Class 4: Deep Learning, Embeddings & Transformers

From Zero to AI Agents: Seven-Class Edition

Scan to open the responder

From Hand-Designed Features to Learned Representations

Class 3 Recap: In Sixty Seconds

- ML **learns rules from examples** when the rules are too complex to write by hand
- Training = **measure loss, adjust parameters, repeat**: gradient descent walking downhill
- **Generalization is the entire game**: a model that only works on its training data is a lookup table with extra steps
- The classical toolbox (trees, forests, boosting) wins on **tabular data**: but you had to engineer the features yourself
- Evaluation is a **business conversation**: precision vs recall, thresholds, decision rights
- And you shipped a **Deep Q-Network** that learned to play Pac-Man: with a neural network inside it you have not opened yet

  Today we do two things: **open up that network**, and remove the biggest limitation of everything we did last class: **the human in charge of deciding what the inputs mean.**

The Limitation of Classic ML

For images and speech, deciding which **input features** to measure can be much of the work.

- In a spreadsheet, we can choose columns such as price, usage, and days since purchase.
- In an image, we start with **pixel brightness**. Which combinations distinguish one shape from another?
- A neural network can learn useful combinations of those inputs during training.

  We still choose the data, architecture, and training objective. The network learns the intermediate features.

Deep Learning Learns Multiple Layers of Representation

```
pixel values → simple patterns → combinations → prediction
```

- A **representation** is a way of describing the input with numbers.
- Each layer transforms the previous layer's numbers into another representation.
- Training adjusts those transformations to make the final prediction more useful.

  Deep means multiple learned layers. We will first see how a small network works, then apply the same ideas to language.

A Neural Network Is Still a Function

A neural network turns **input numbers into output numbers** using adjustable weights and biases.

- **Architecture:** which neurons connect to which other neurons.
- **Parameters:** the weights and biases on those calculations.
- **Training:** change the parameters to reduce loss on examples.
- **Inference:** use the learned parameters to predict for a new input.

  Keep asking: **what goes in, what comes out, and which numbers change during training?**

Today’s Roadmap

1. **One neuron:** inputs, weights, bias, activation.
2. **A network:** connect neurons and learn intermediate features.
3. **Training:** predict, measure loss, calculate gradients, update.
4. **Experiment:** compare a straight boundary with a nonlinear network.
5. **Language:** tokens, embeddings, attention, and transformers.
6. **Practice:** inspect and train the tiny language model.

Neural Networks

The Artificial Neuron

An artificial neuron is a small calculation, loosely inspired by biological neurons.

1. Receive **inputs**: numbers from the data or an earlier layer.
2. Multiply each input by its **weight**, then add the results.
3. Add a **bias**: a learned offset.
4. Apply an **activation function** and pass the output onward.

One neuron makes one feature. Connecting many neurons lets the model combine features.

A Single Neuron in Action

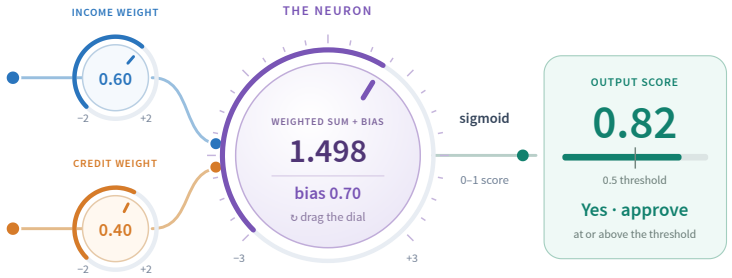
TURN THE DIALS. WATCH THE DECISION CHANGE. Illustrative loan example · scaled inputs

Income 0.85

scaled input · drag to change

Credit 0.72

scaled input · drag to change



income × weight **0.85 × 0.60** = 0.510

+

credit × weight **0.72 × 0.40** = 0.288

+

bias **0.70** shift the result

=

sum **1.498** before activation

→

sigmoid(sum)**0.82**output score

Weights scale the inputs. Bias shifts the sum. Activation turns it into a score.

Pause flow | Pause dial | Reset

One neuron, one decision: take inputs, multiply by weights, output a score. Score > 0.5 → Yes. Score < 0.5 → No. That's classification. One neuron isn't smart: but **millions together** can recognize faces, translate languages, and write code.

The Neuron as a Formula

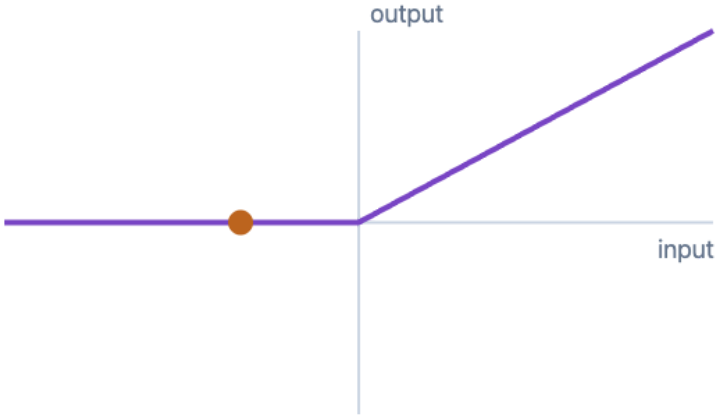
$$\text{output} = \text{activation}(\text{weight}_1 \times \text{input}_1 + \text{weight}_2 \times \text{input}_2 + \text{bias})$$

- A **weight** controls how an input changes this neuron's calculation. It can be positive, negative, or near zero.
- The **bias** shifts the calculation, like the intercept in a regression.
- The **activation** bends the response. The next layer receives this result.

  In the previous demo, change just one weight. Then change just the bias. **Which part of the formula moved?**

Why Nonlinearity Matters

Move the input across zero. Compare a straight response with a response that bends.



Activation ReLU

Input

Input **-1.00**

Output **0.00**

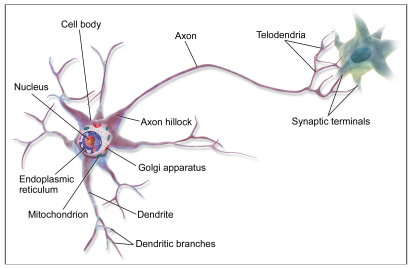
ReLU: zero below zero, unchanged above zero.

Tanh: a smooth bend with output between −1 and 1.

Stacking only linear layers still gives a linear function.

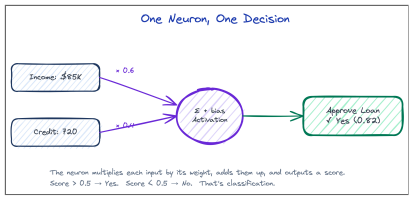
Nonlinear activations let layers combine into curved decision boundaries.

Inspired by the Brain



Biological neuron

Biological: receives and sends electrochemical signals.



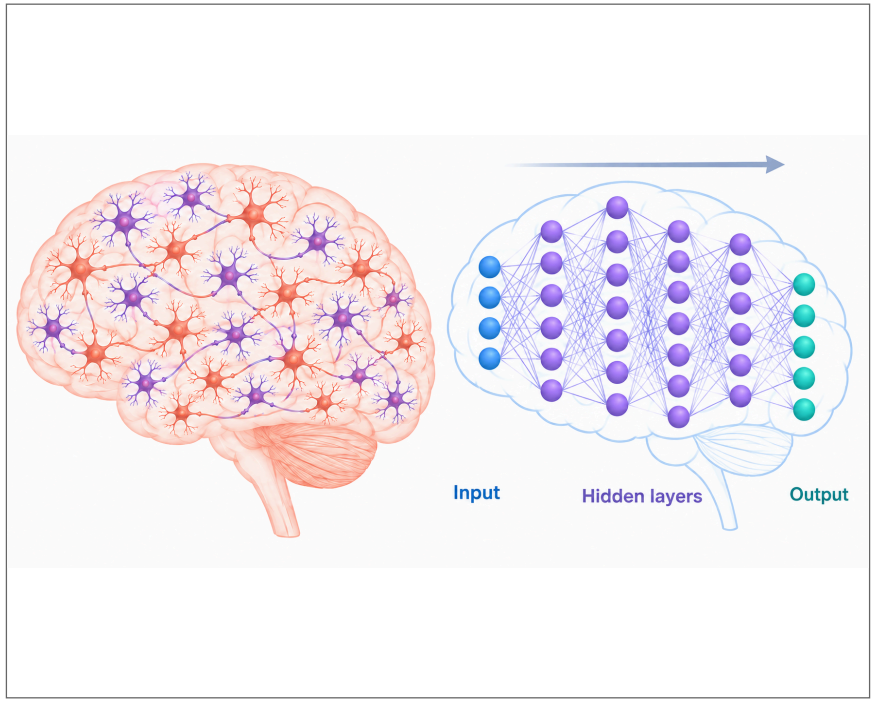
Artificial neuron

Artificial: combines numbers with weights, bias, and an activation.

The connection is an inspiration. This simplified mathematical model does **not** reproduce all the ways a brain learns.

A Brain and a Neural Network

Biological brain **Artificial neural network**

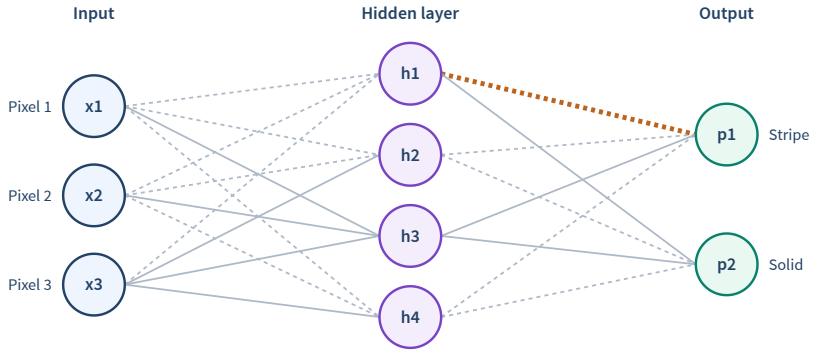


The shared idea: **many connected units passing signals.**

Biological neurons inspire the idea. Artificial neurons are simplified calculations.

One Neuron → A Network

Our small network reads three pixel values and predicts **stripe** or **solid**. Trace its connections.



3 inputs × 4 hidden neurons + 4 hidden × 2 outputs = **20 connection weights**

Hidden layer → Reset

Click Hidden layer to follow the input through the weighted connections.
0 = dark pixel · 1 = bright pixel · calculated probabilities · each slide starts from the same weights.
Hidden just means between input and output. Those neurons produce intermediate numbers, not answers we label by hand.

How a Neural Network Learns

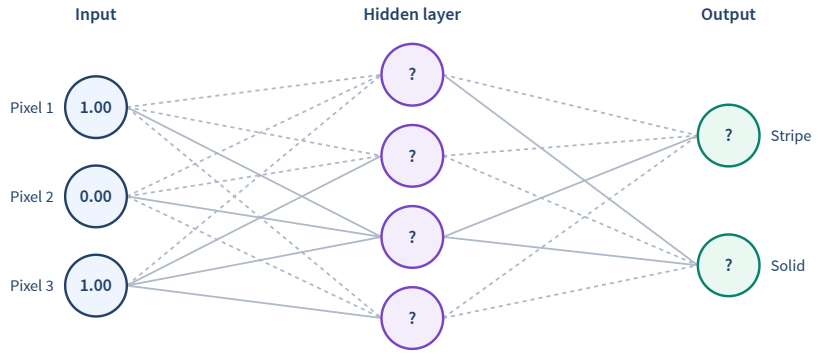
Imagine a row of adjustable dials. Together, their settings determine the prediction.

1. **Predict:** pass an example through the network with the current settings.
2. **Measure loss:** compare the prediction with its training target.
3. **Backpropagate:** calculate how a small change to each setting would change loss.
4. **Update:** use those gradients to adjust the weights and biases.
5. **Repeat:** use many examples, then check held-out examples.

Backpropagation measures **sensitivity**. The optimizer uses that information to move the dials.

The Training Loop

Step through one complete training update. Watch when the parameters actually change.



Target: **stripe**Target probability: **0.364**Loss: **1.010**Updates: **0**

Predict → Reset

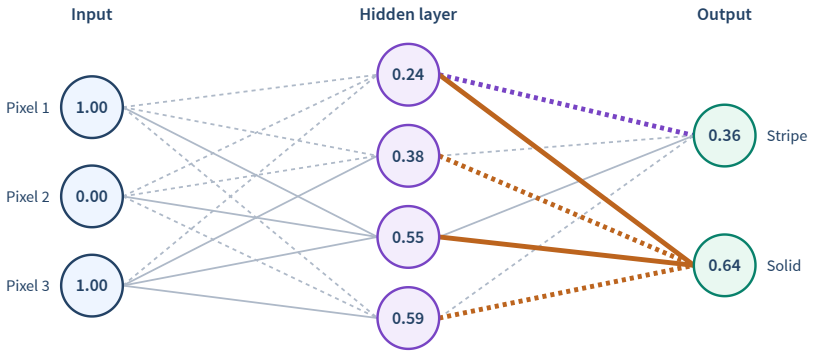
Ready: run a training example through the network.

0 = dark pixel · 1 = bright pixel · calculated probabilities · each slide starts from the same weights. Orange paths: prediction > 0.5.

The forward pass, loss, and backward pass use the current weights. **Only the update step changes them.**

What's Actually Being Adjusted?

Choose a connection and move its weight. The input stays fixed; the prediction changes.



Target: **stripe**Target probability: **0.364**Loss: **1.010**Updates: **0**

Parameter Hidden 1 → output 1

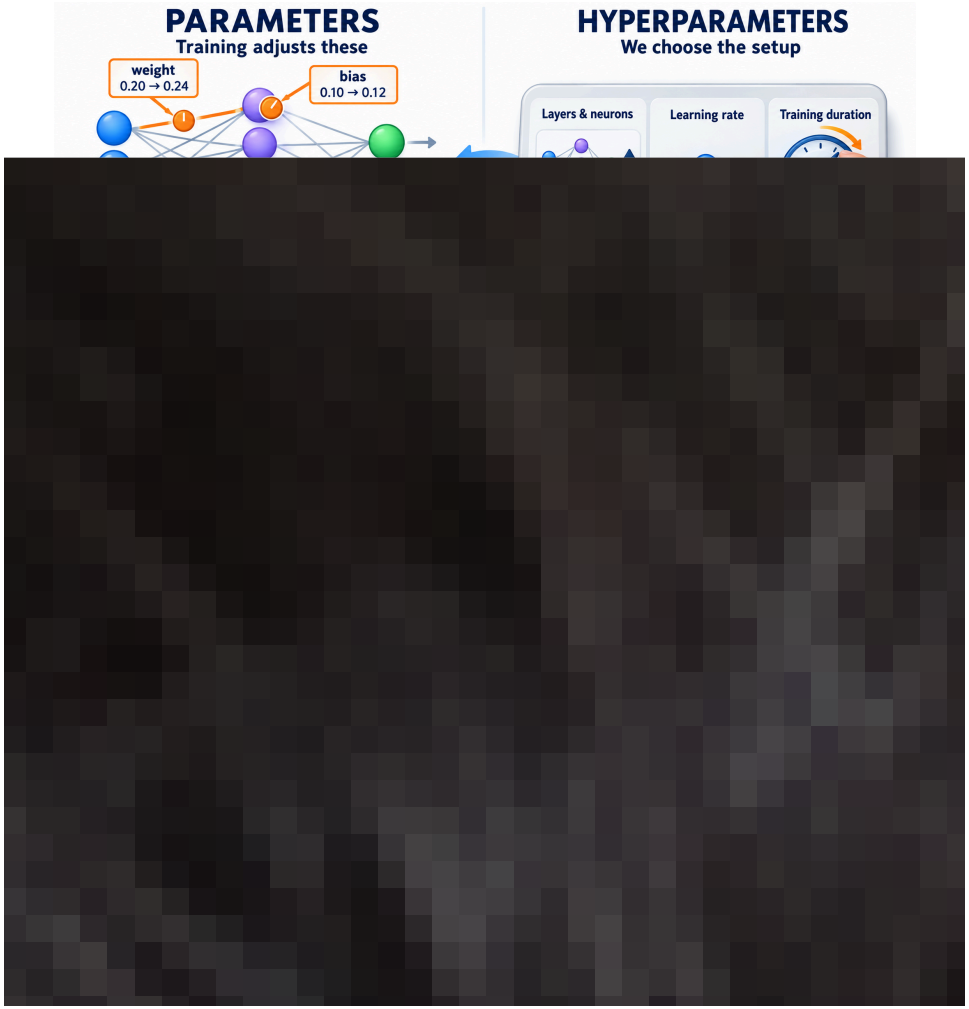
Value -0.26

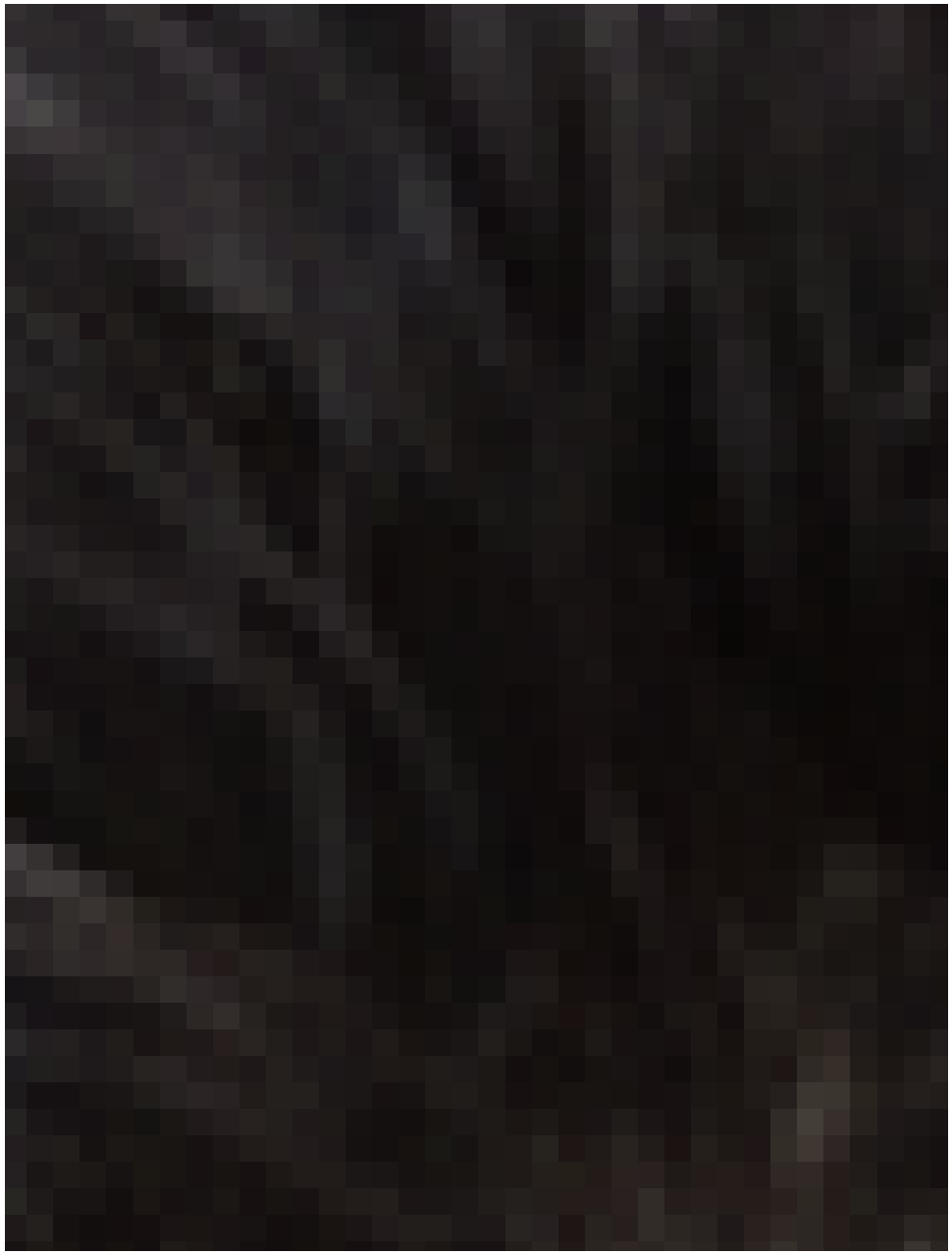
Reset

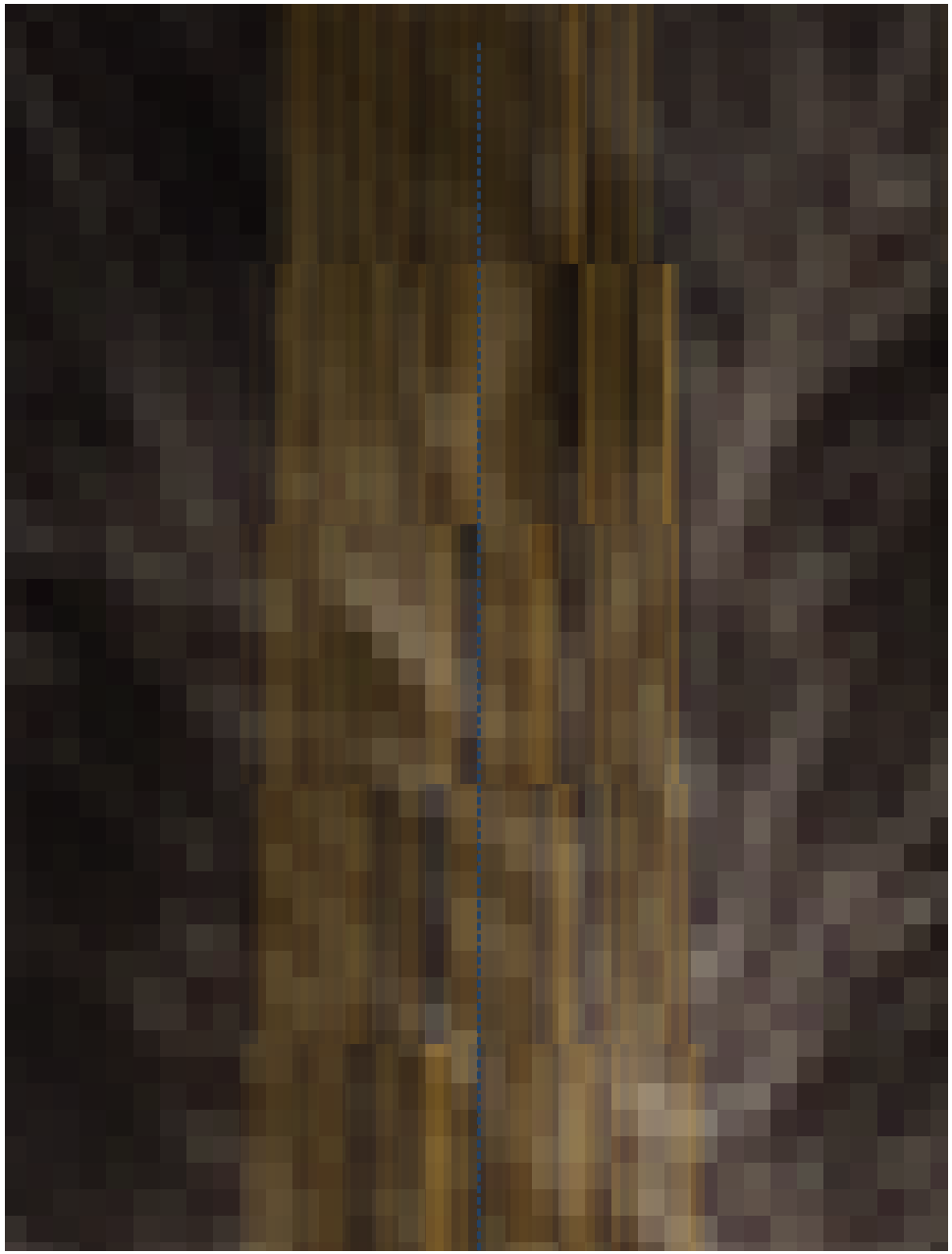
Hidden 1 → output 1 = -0.262. A negative weight subtracts from the receiving neuron's sum for this nonnegative input.
0 = dark pixel · 1 = bright pixel · calculated probabilities · each slide starts from the same weights. Orange paths: prediction > 0.5. Purple: selected parameter.

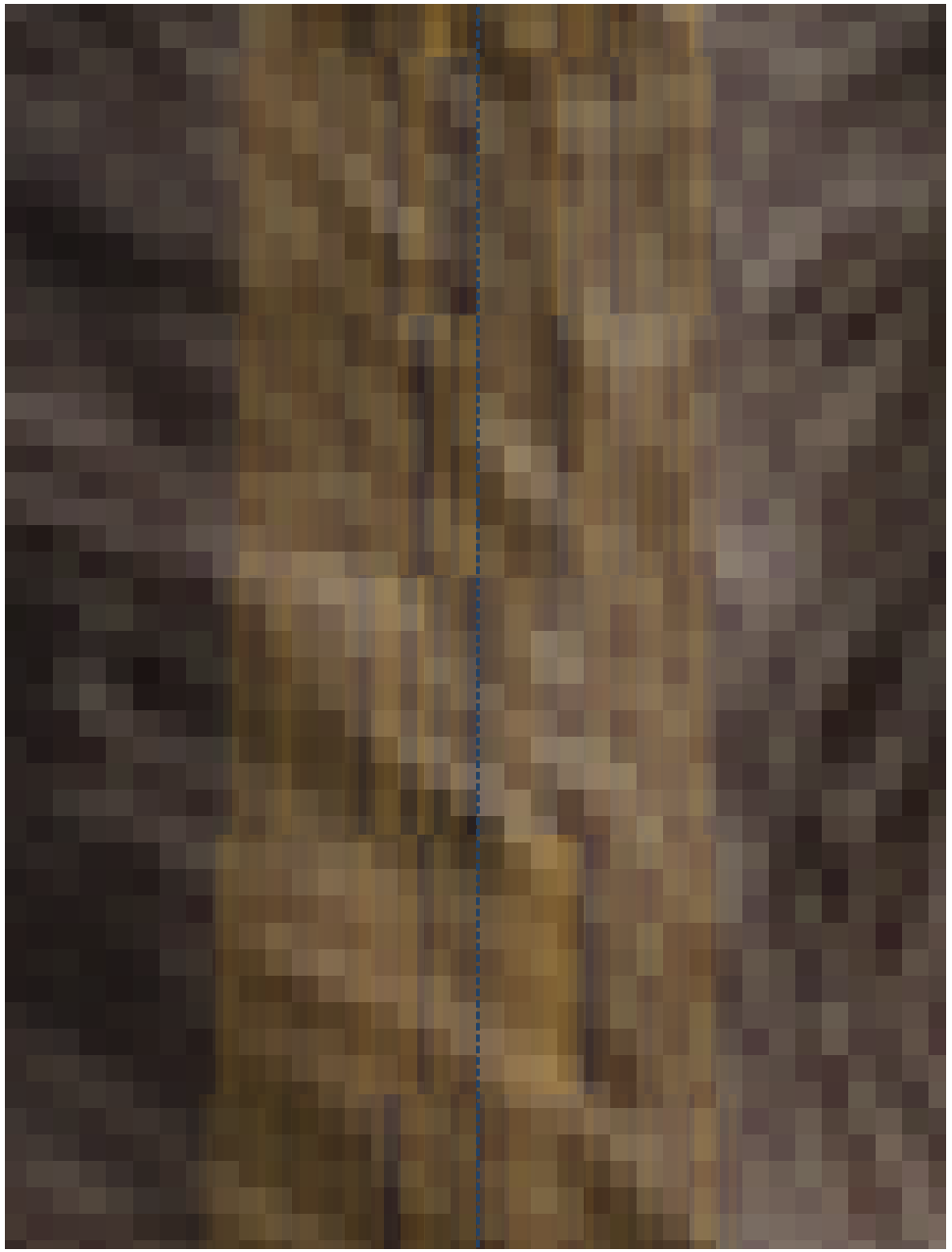
This network has **20 connection weights + 6 biases = 26 learned parameters**. Inputs are data; neuron outputs are calculated values.

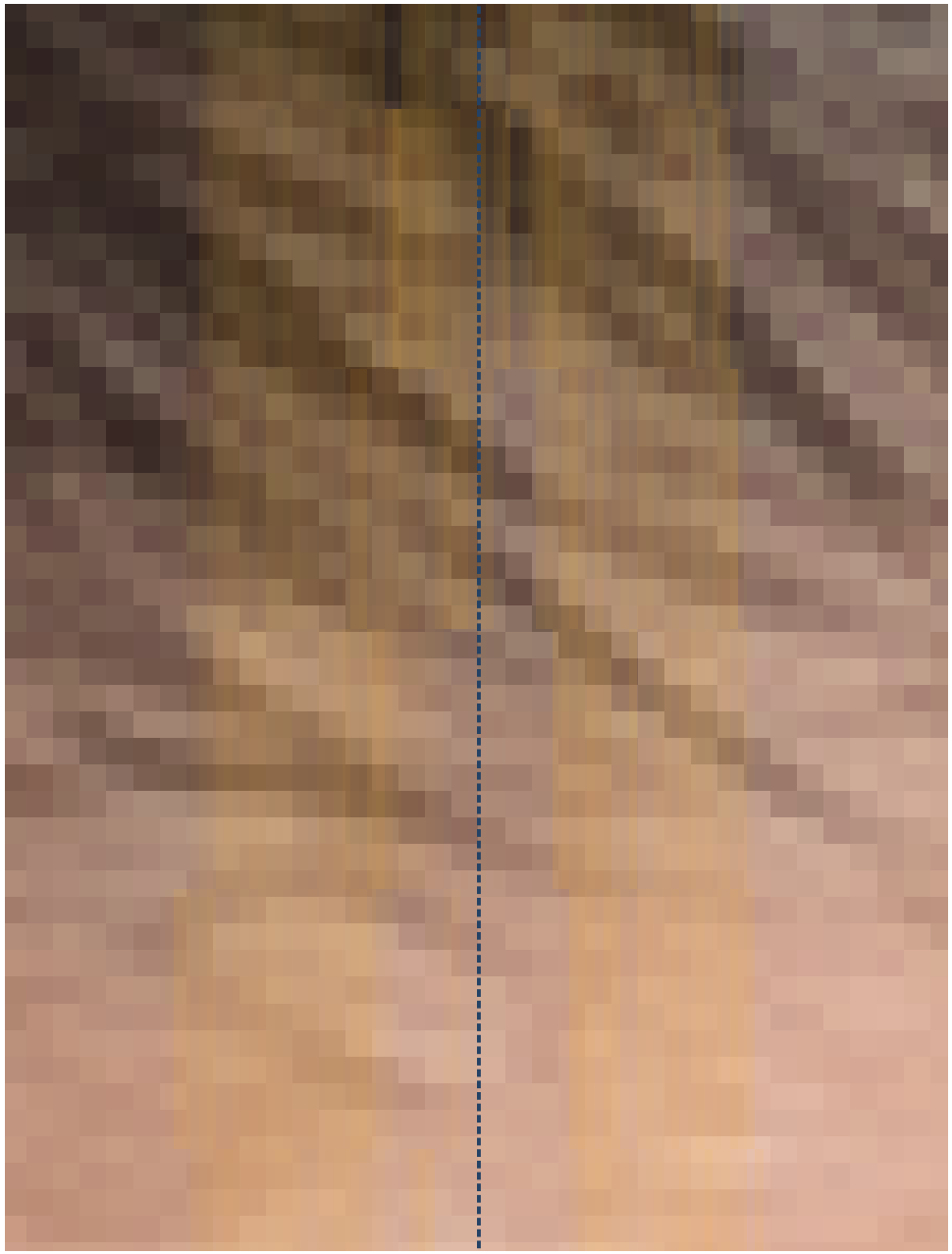
Parameters vs Hyperparameters

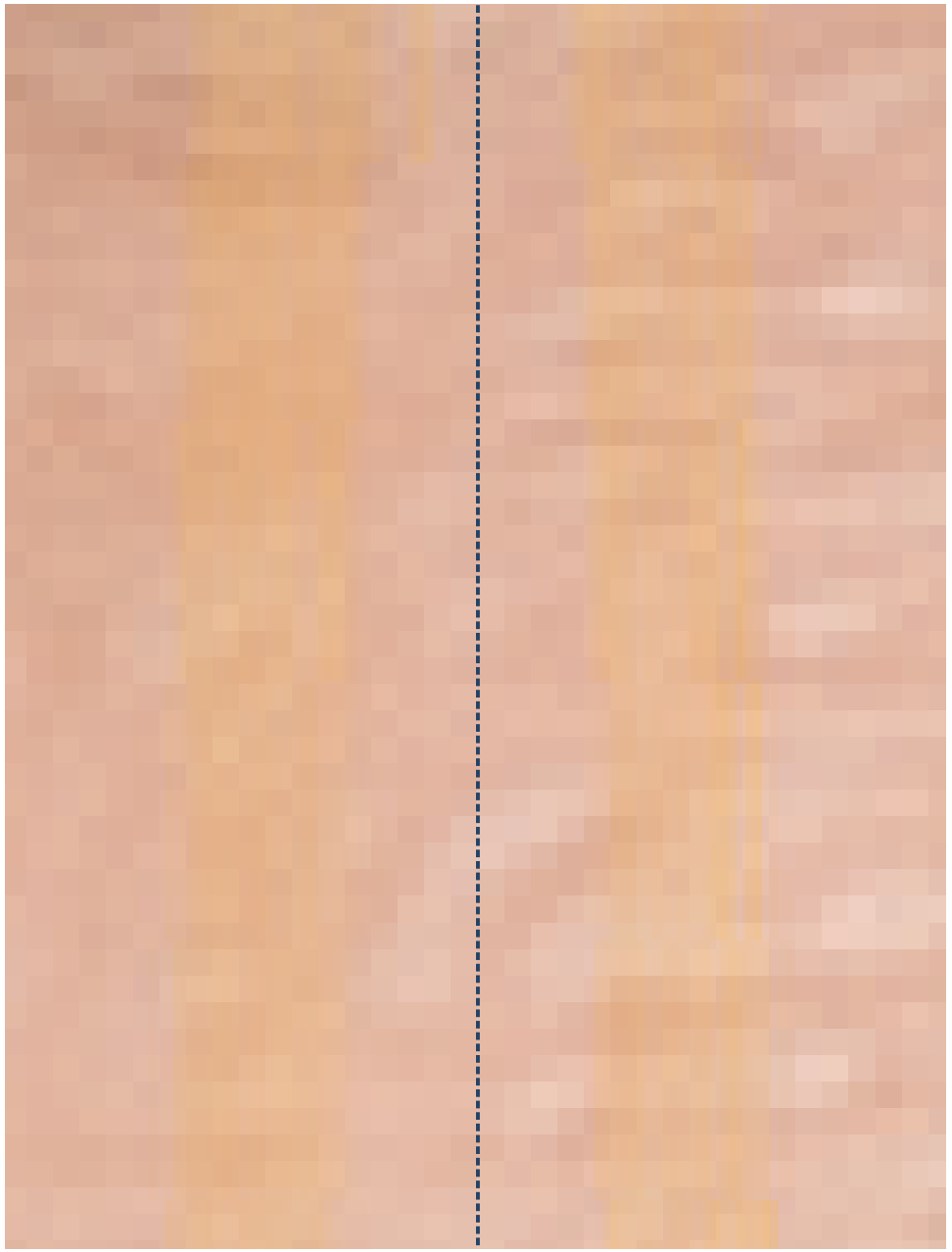


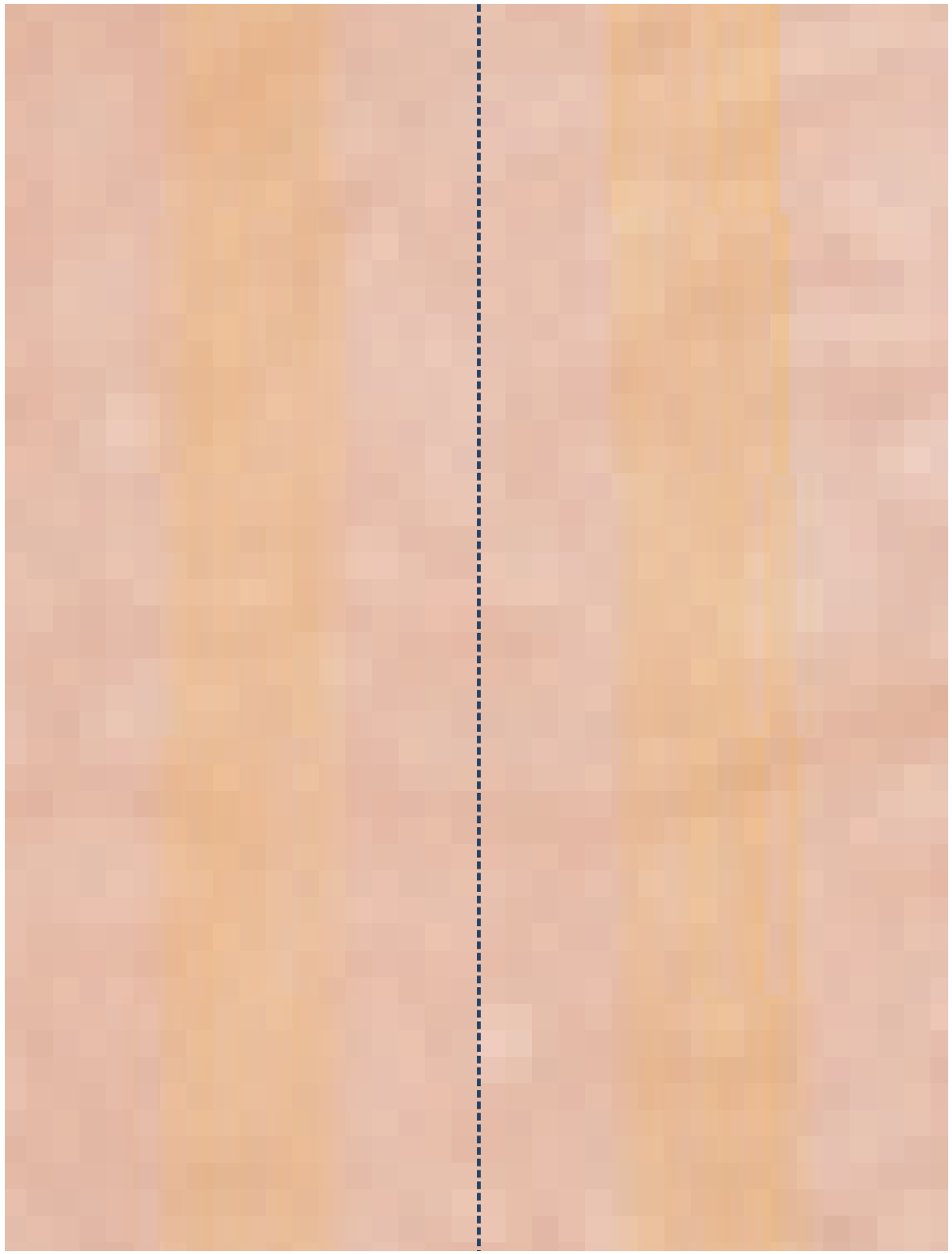


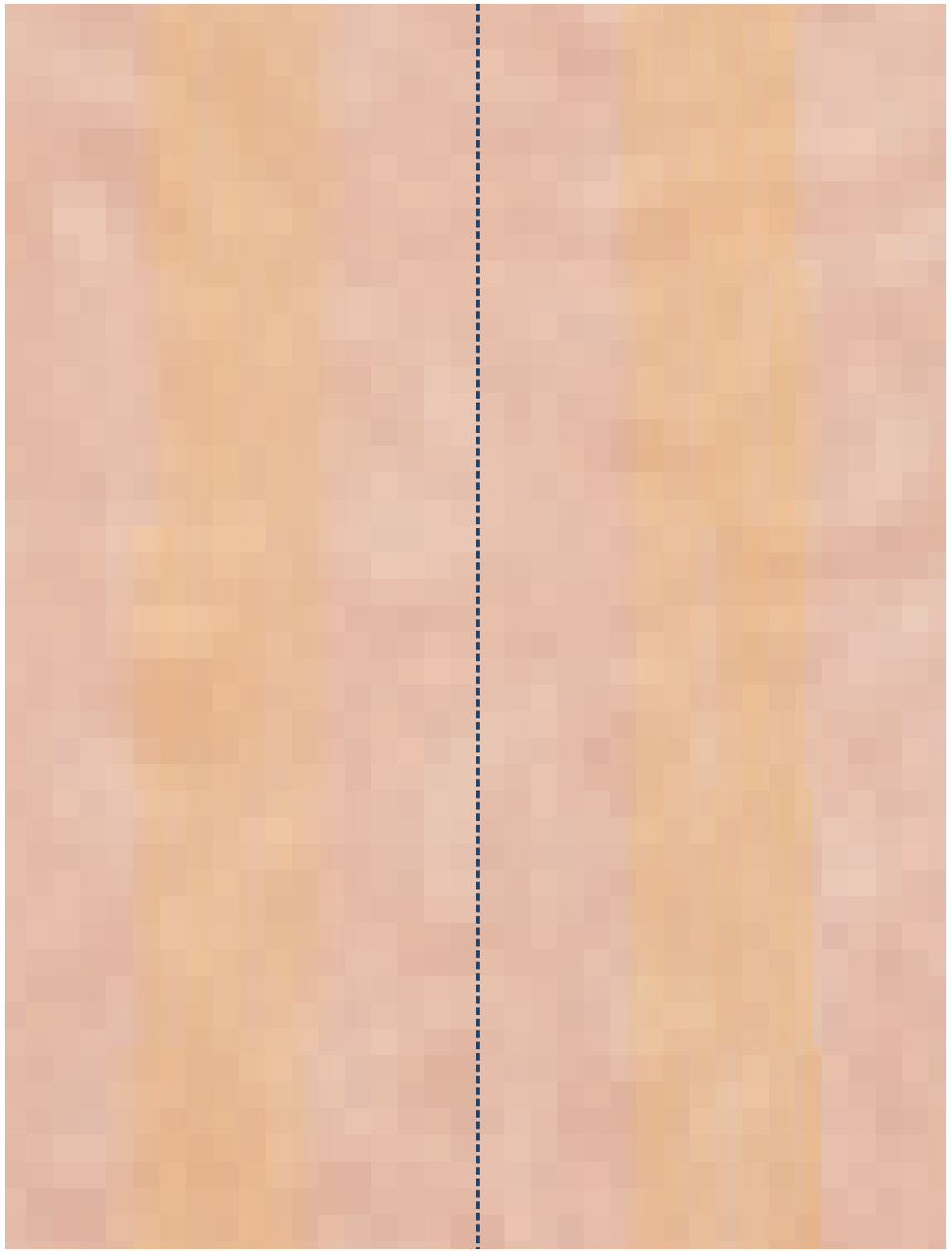


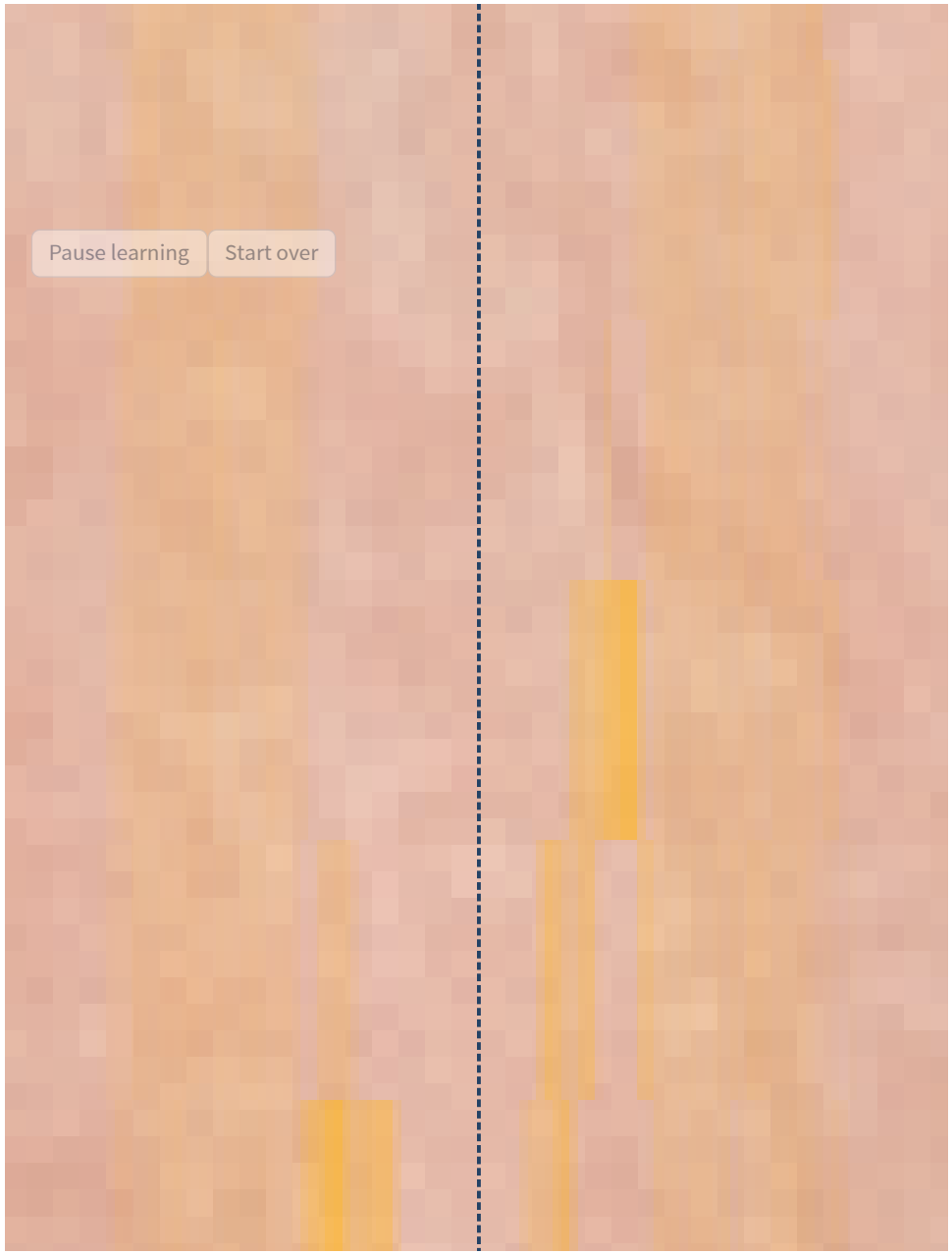


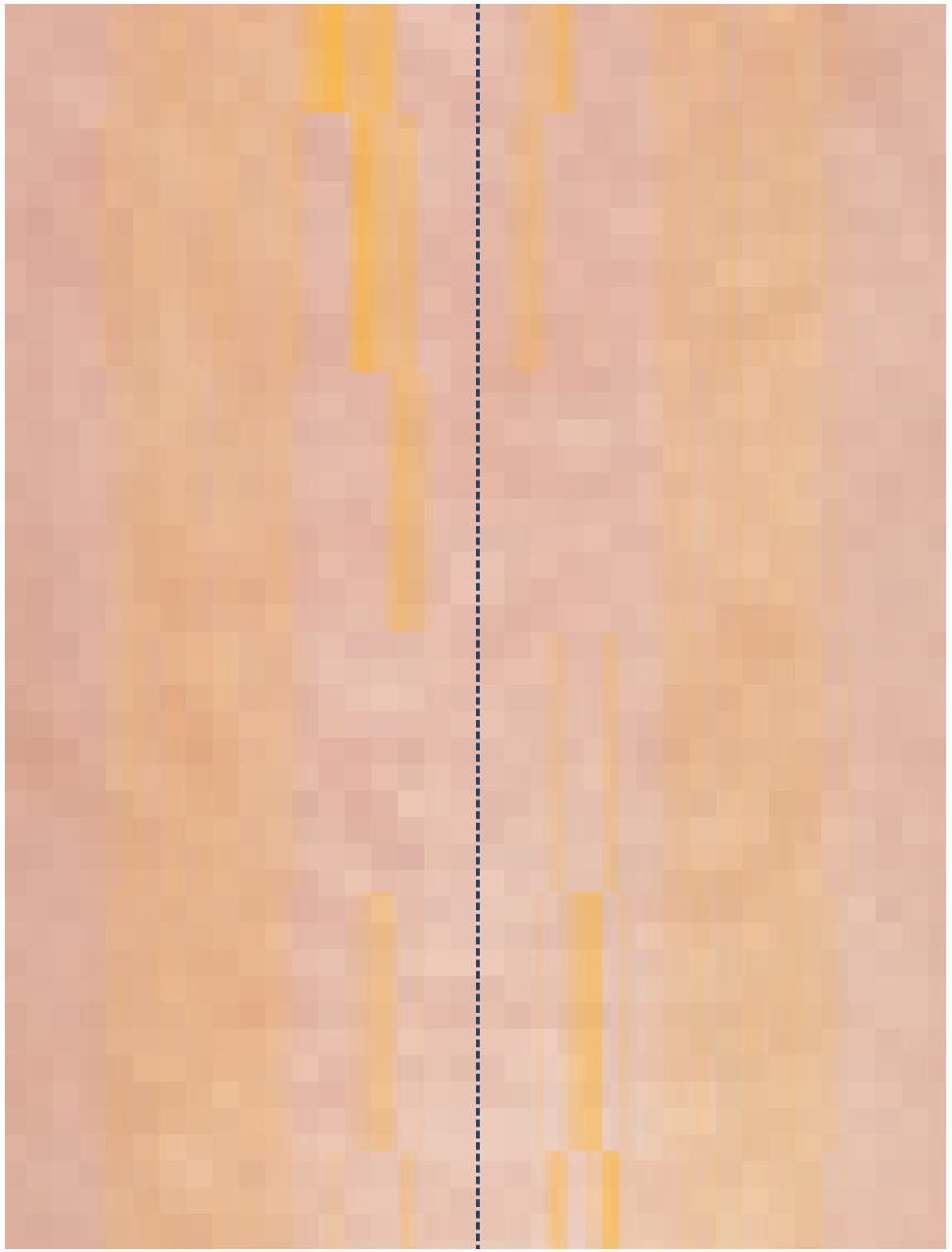


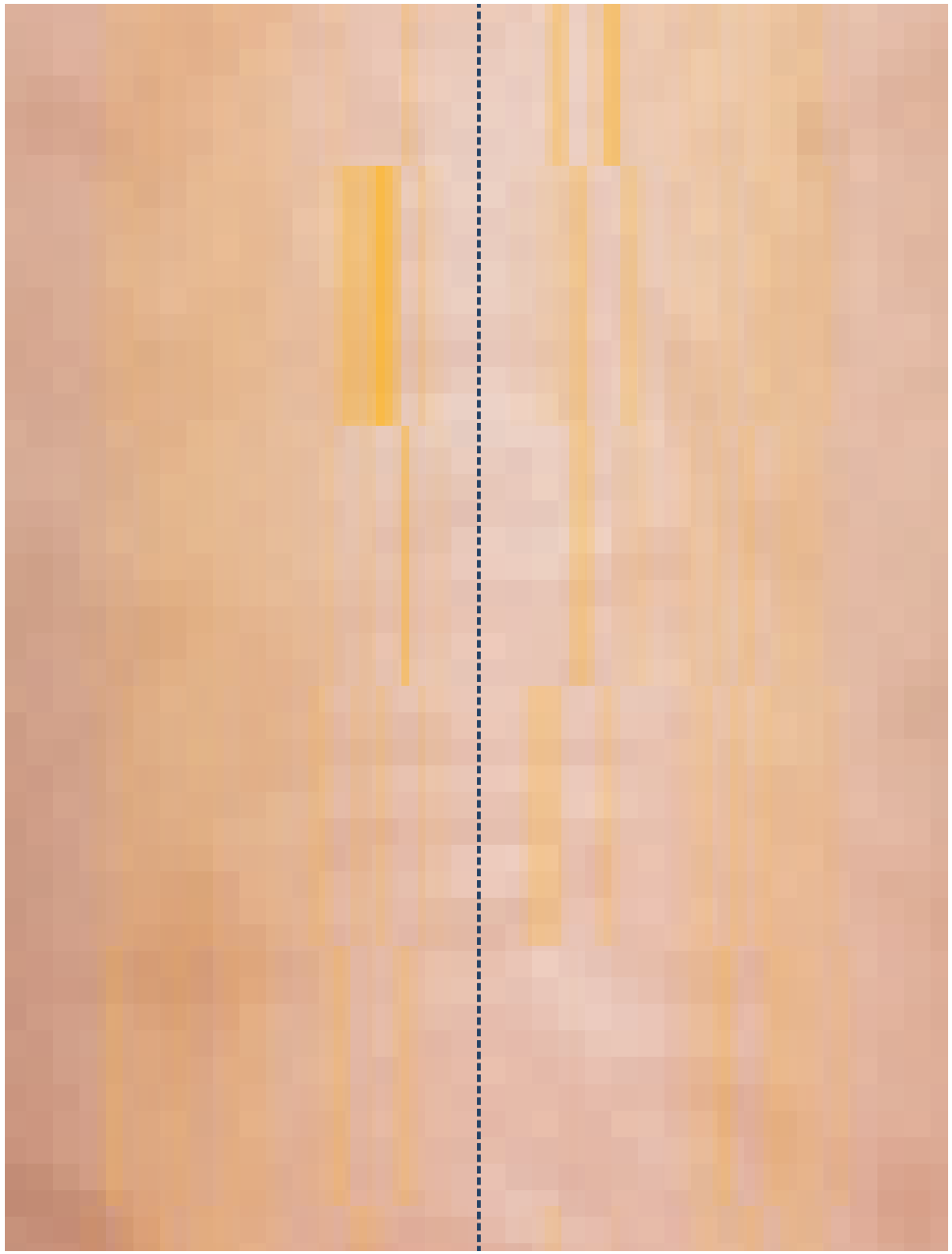


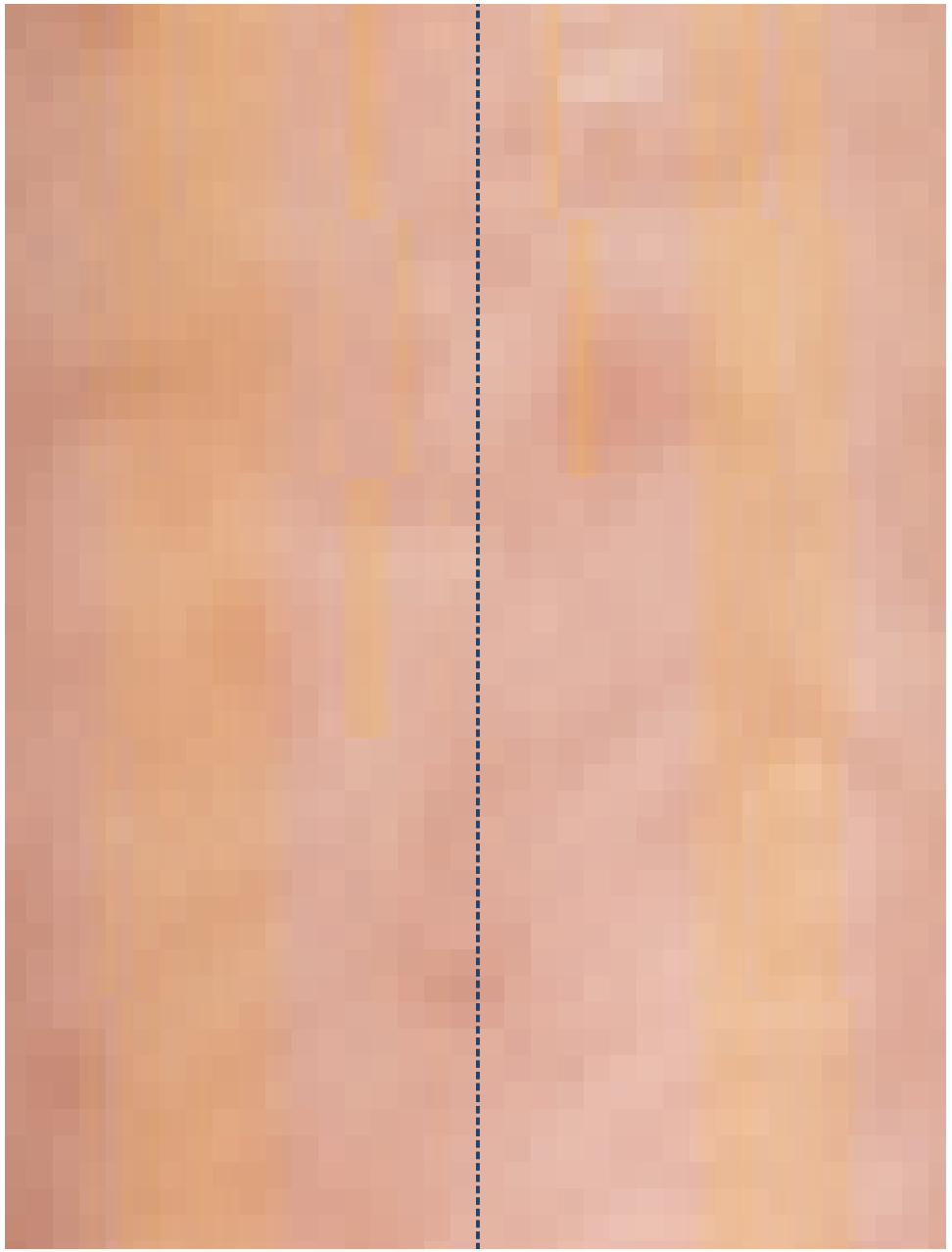


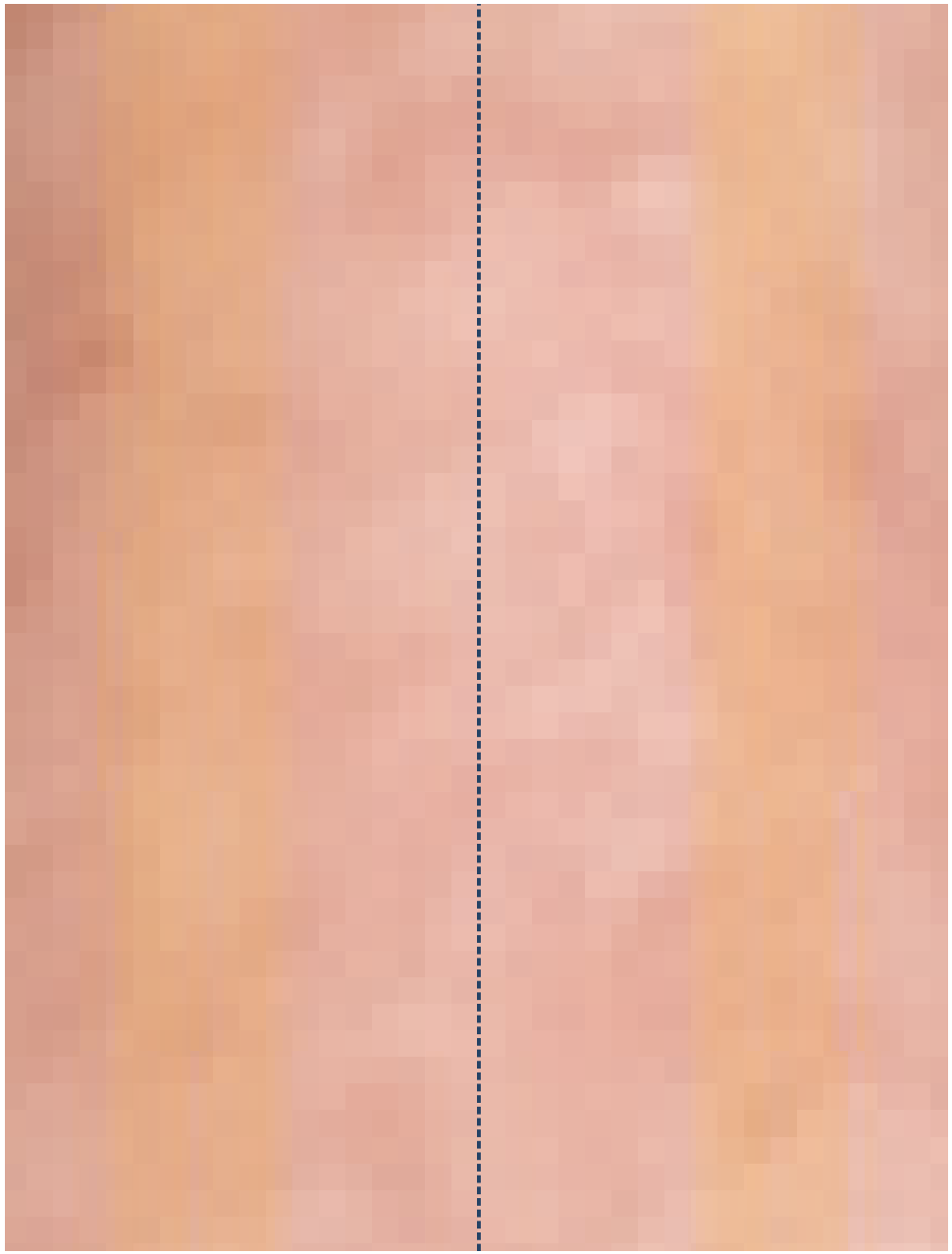


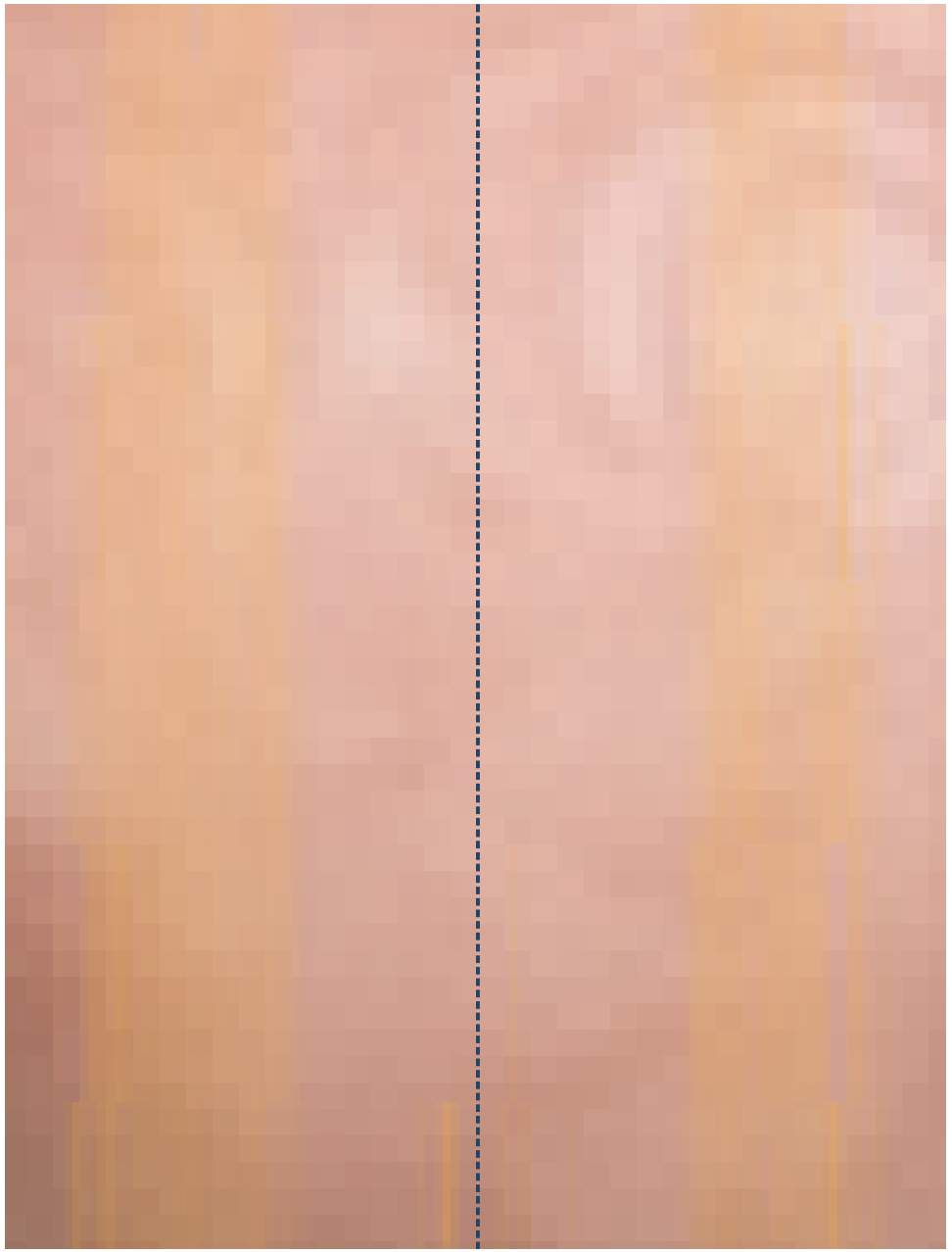

Pause learning
Start over

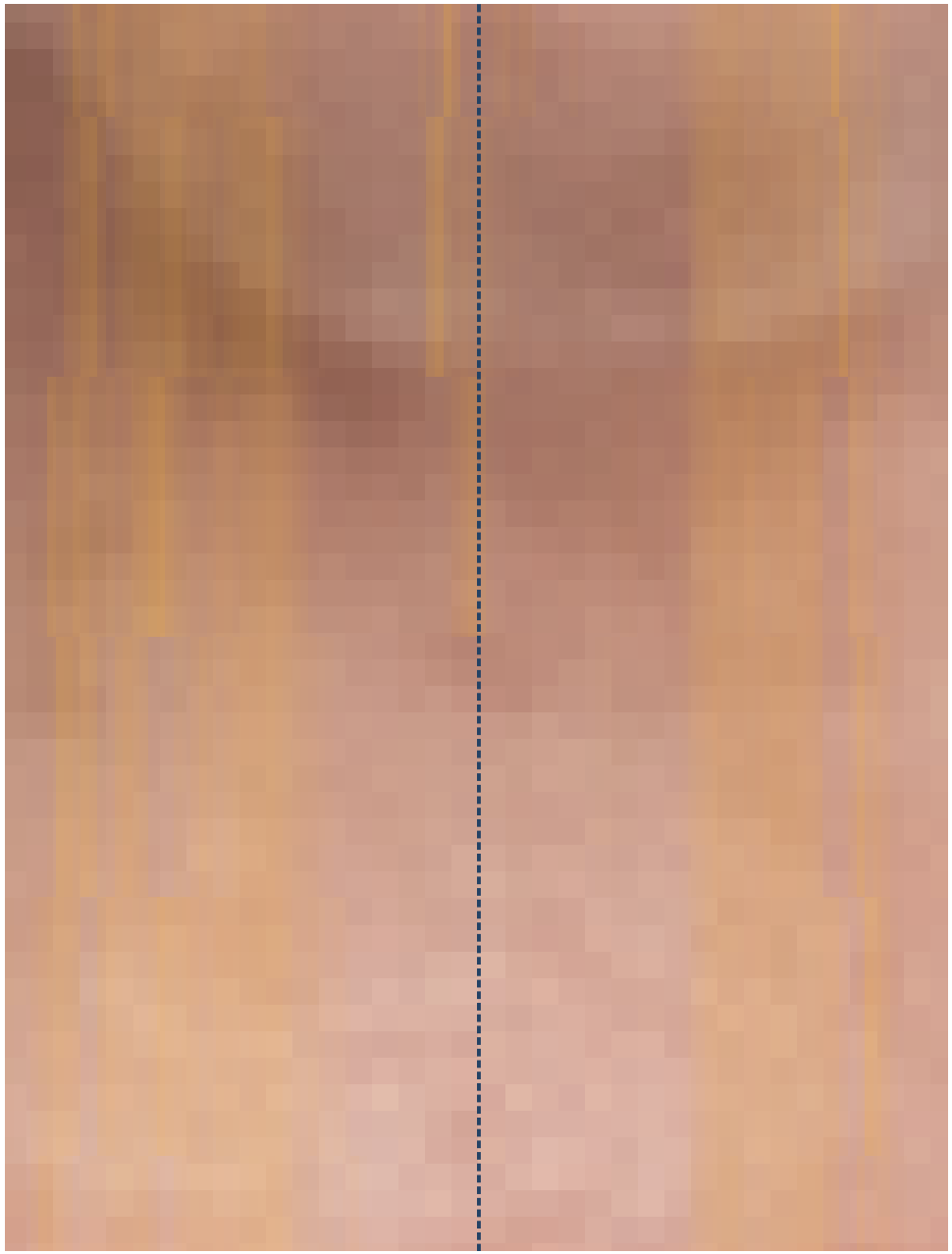


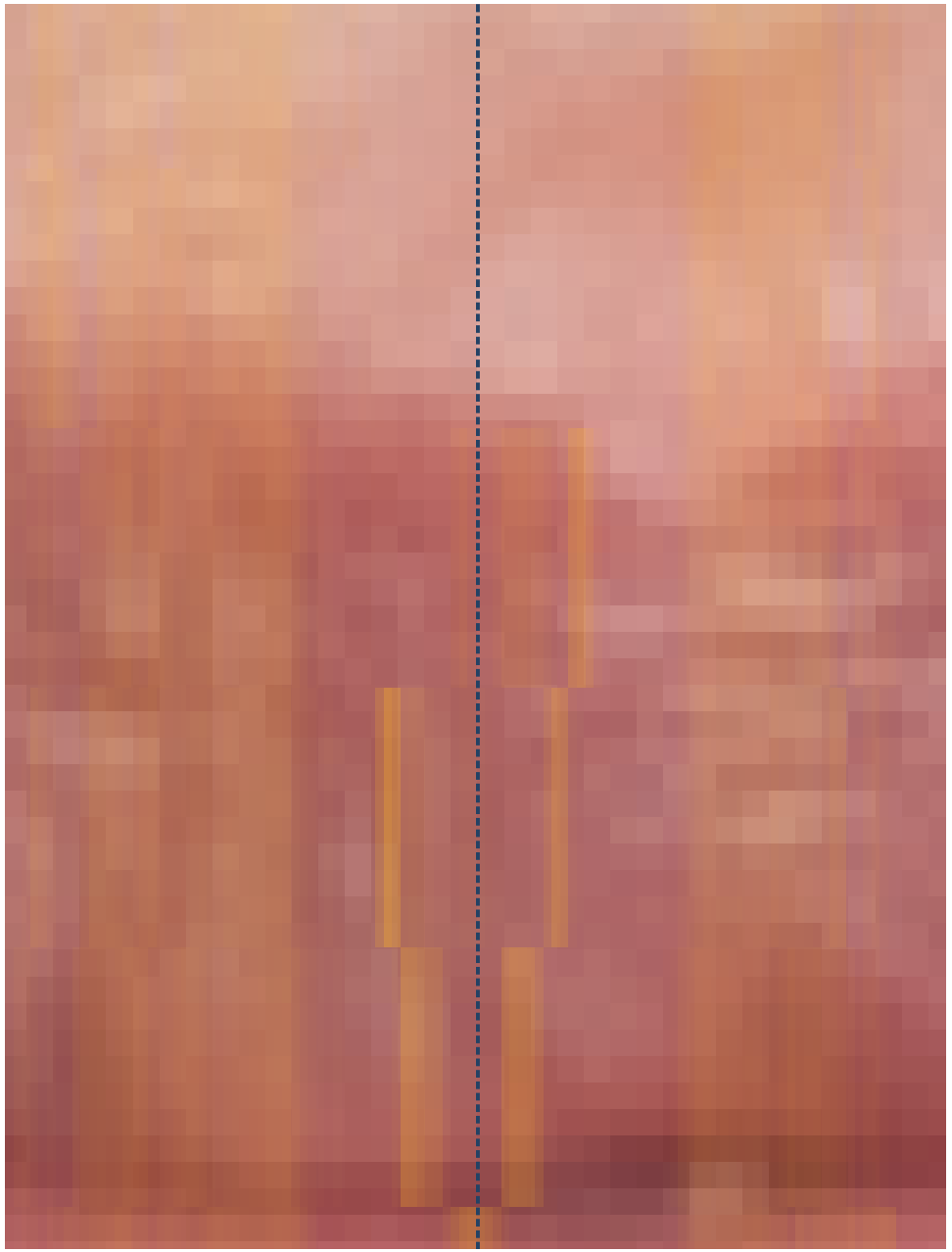


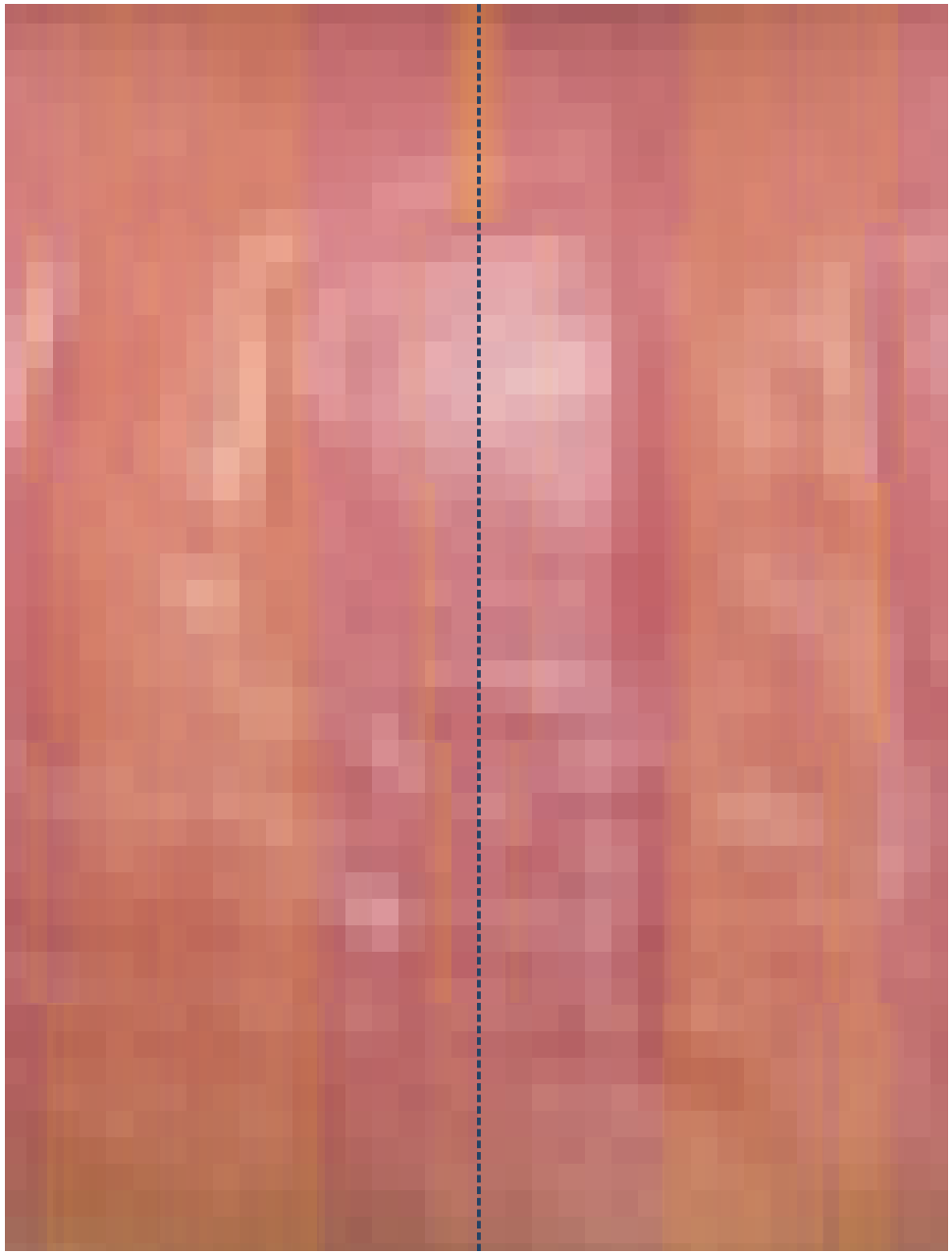








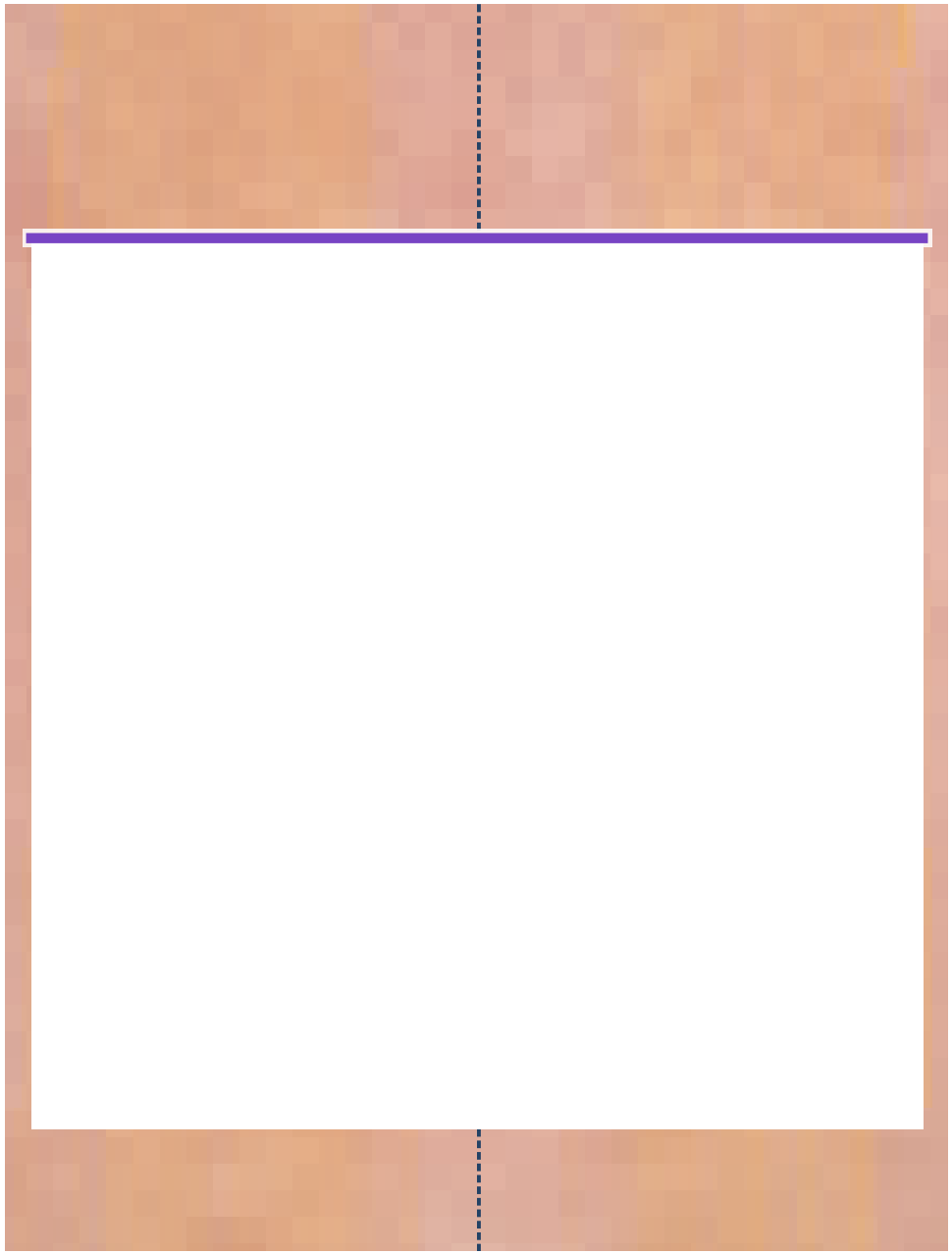


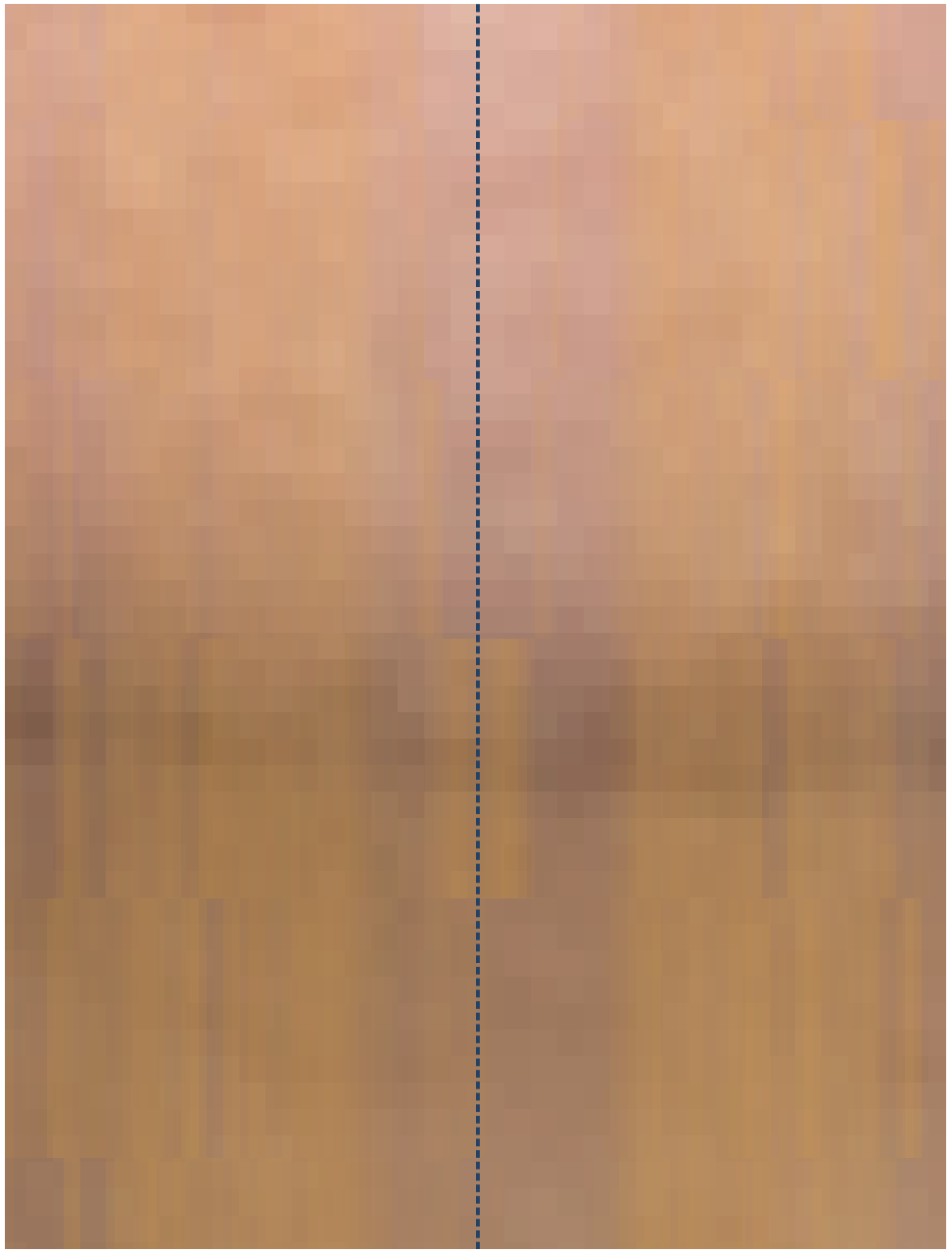


- Batch size 10 with 100 examples means **10 updates per epoch**.
- At the next epoch, the model sees the examples again with new parameter values.
- More epochs can reduce training loss while making performance on new examples worse.

Return to the circle experiment. Compare training loss and held-out loss after more epochs. **Did both improve?**

Tip





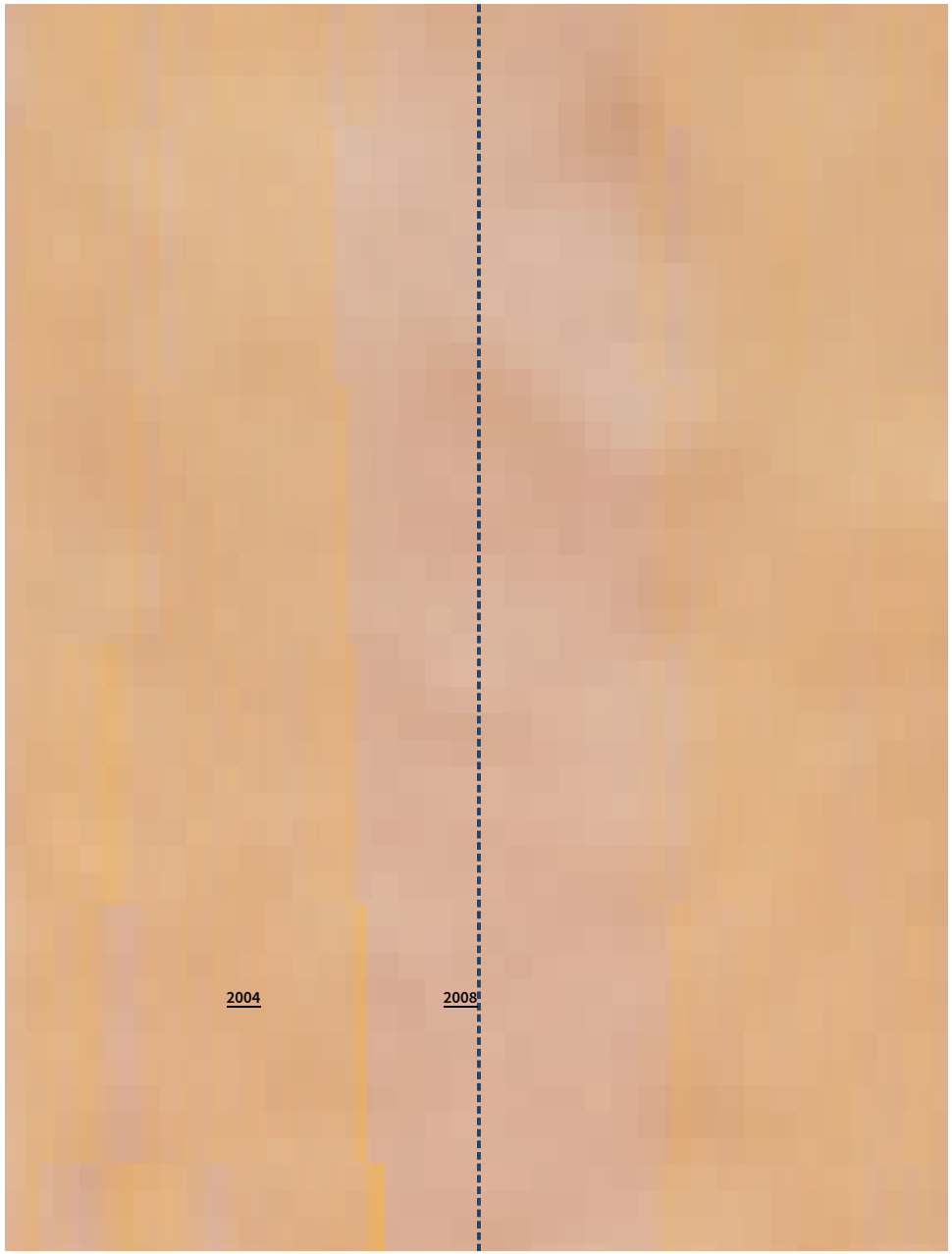
The **same filter weights** are used at every position. Training learns the filters; sliding a trained filter is part of the forward pass. Pac-Man's CNN uses this idea to process the game screen.

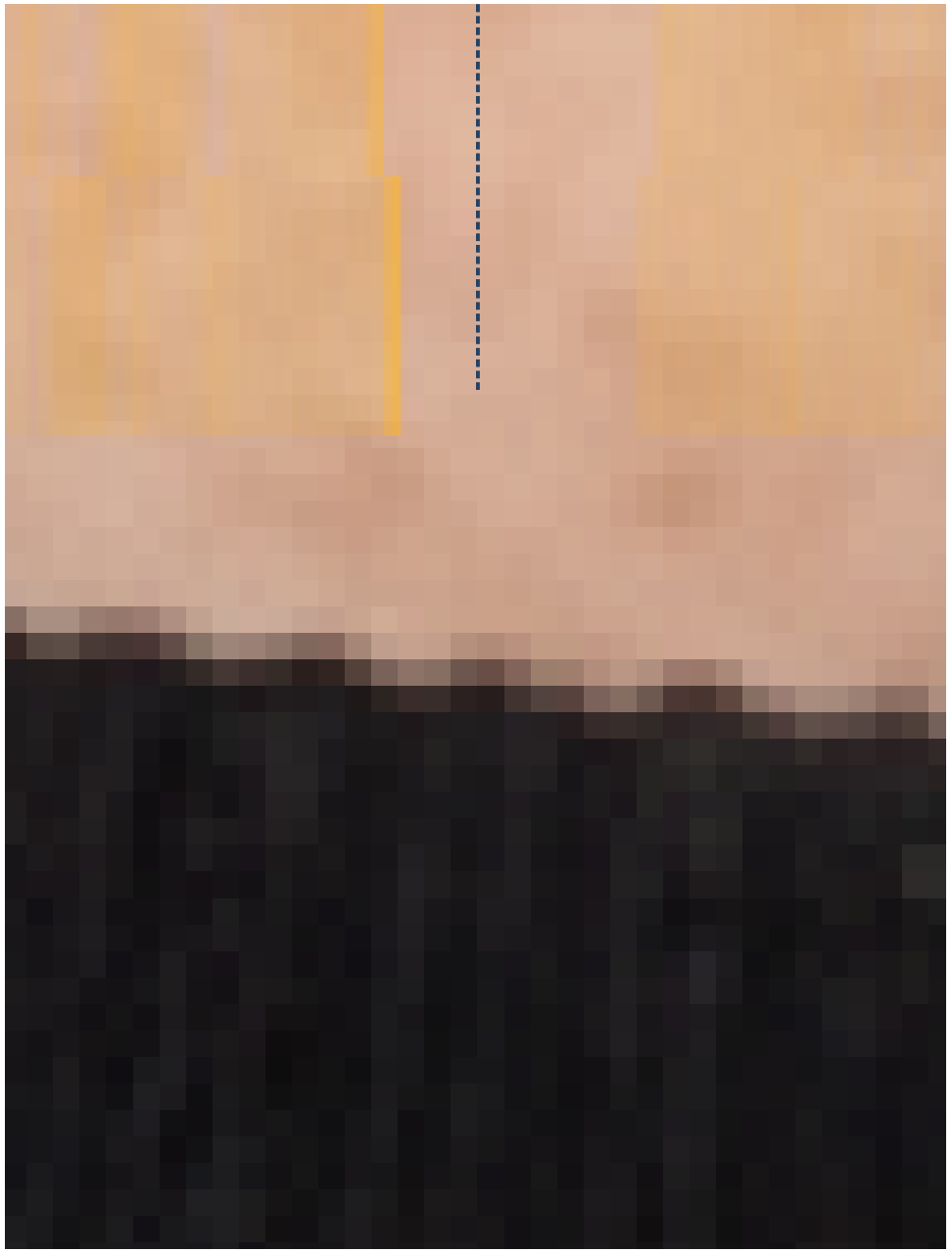
- A layer can be calculated as a **matrix multiplication**, followed by bias and activation.
- Many of those calculations can run at the same time.
- **GPUs** provide hardware for doing large groups of numerical operations in parallel.

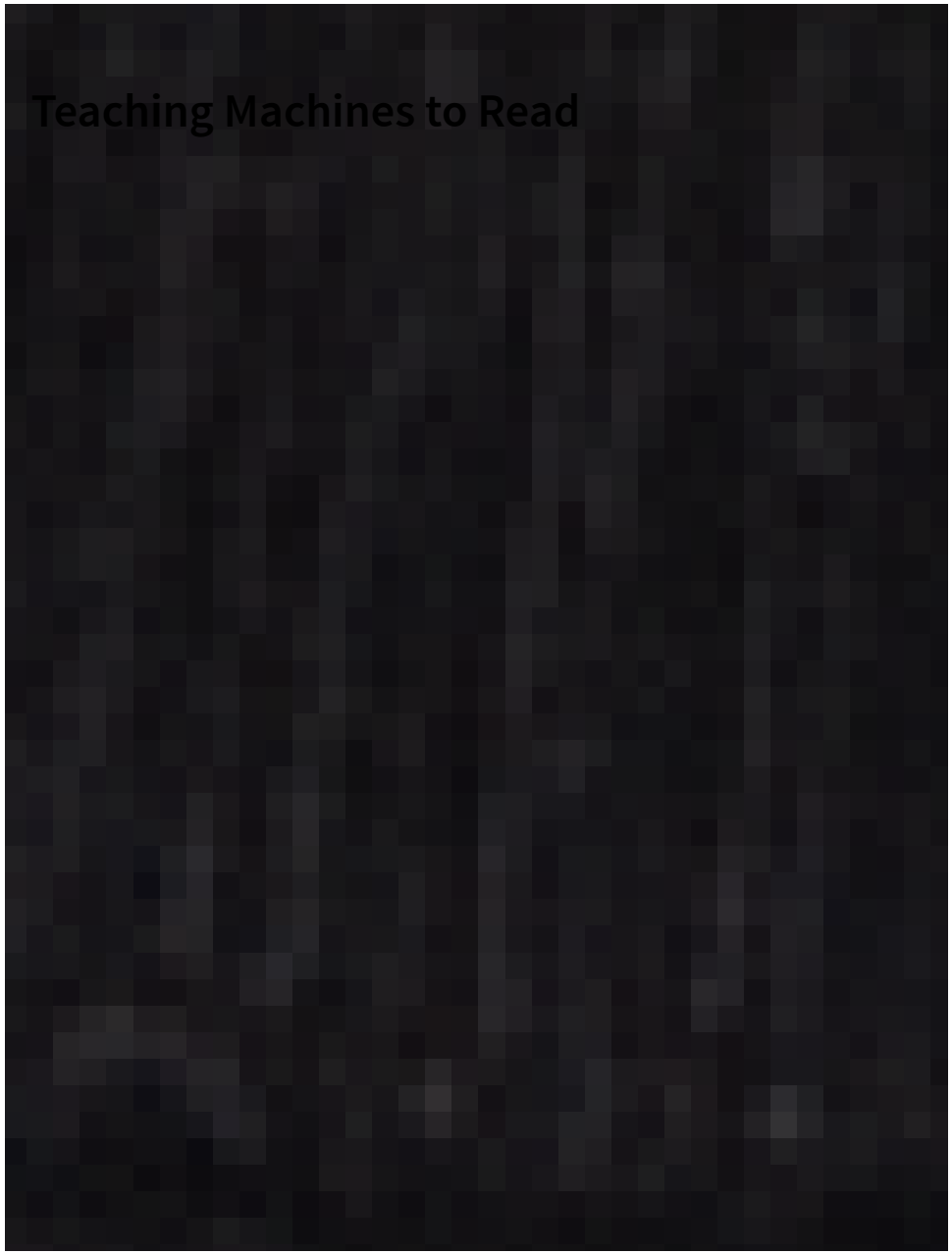
The small network we just trained uses the same basic learning loop. A larger network needs more memory, computation, and data.

Watch the Cortical Labs demonstration ↗

Doom experiment and code Schultz, Dayan & Montague, 1997


2004
2008





Why Language?

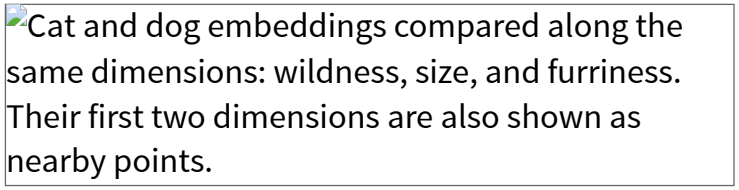
So far, every model we've seen works with **numbers**: house prices, pixel values, game screens.

- But the biggest AI breakthroughs came from teaching machines to read **text**
- Why? The internet is mostly text. Trillions of words, freely available
- If you can teach a machine to read, you can train it on the **entire internet**

> **Note**
>
> This is exactly what happened. The shift from images to language is what gave us ChatGPT, Claude, and every modern AI assistant.

The Problem: Words Aren't Numbers

How do you feed words into a neural network that only understands numbers?
An **embedding** is a learned **vector** (a list of numbers) that represents a word or token. Training adjusts these numbers to help the model make predictions.


Cat and dog embeddings compared along the same dimensions: wildness, size, and furriness. Their first two dimensions are also shown as nearby points.

> **Warning**
>
> Both words use **the same dimensions, in the same order**. These values and labels are simplified examples; real dimensions are learned and blend many concepts. Words used in similar contexts tend to have similar embeddings.

Words as Points in Space

Word2Vec (2013) learned that words in similar contexts get similar vectors.
"The ___ sat on the mat" → cat, dog, child all fit → similar vectors.

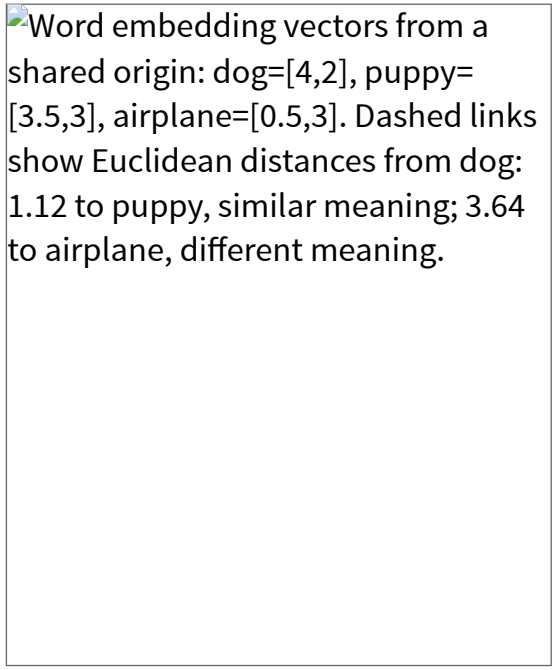
Words with similar meanings cluster together. Relationships like gender, royalty, and age become **directions** you can navigate with math.

> **Important**
>
> **king - man + woman ≈ queen**
>
> The model learned gender as a direction in vector space, without anyone teaching it!

Distance Reflects Meaning

An **embedding** represents a word as a vector. Nearby vectors can capture related meanings.

- **“Dog” and “puppy”** have related meanings, so their vectors are close together.
- **“Dog” and “airplane”** have less in common, so their vectors are farther apart.
- The dashed lines measure **distance between vectors**. A smaller distance represents greater similarity in this example.



Illustrative 2D embeddings. Distances calculated from the coordinates shown.

> **Tip**
>
> **Meaning becomes something we can compare with math:** “dog” is closer to “puppy” than to “airplane.”

Explore Word Embeddings in 3D

> **Tip**

> **Word2Vec 10K** loads automatically. Drag to rotate, scroll to zoom, and search for “king” to see nearby words. Try toggling between PCA and t-SNE projections.

Context Determines Meaning

One vector per word has a ceiling: “bank” in a finance memo and “bank” in a river report **should not mean the same thing**.

Word2Vec gives both banks the same point in space: a learned representation based on the contexts in its training corpus. Modern models instead create **contextual representations**: the vector for “bank” is adjusted, on the fly, by the words around it. The mechanism that does that adjusting is **attention**: coming right up.

Context Moves “Bank” in Embedding Space

Illustrative 3D positions. **+ means adding context**, not exact vector addition.

How Words Become Tokens

A **tokenizer** breaks text into pieces called **tokens**. BPE (byte-pair encoding) learns which pieces to combine.

Illustrative split, IDs, and vectors. Each model uses its own fixed tokenizer.

One word, three tokens: each takes context space and can contribute to API cost.

Talking in Tokens

Tokenization splits text into pieces and **preserves the original wording**.

Image: **3Blue1Brown**

Sequences & Attention

The Problem with Order

“Dog bites man” and “man bites dog” use the same words but mean different things.

- A representation that only counts words loses their order.
- A network can use order if the input and architecture preserve it.
- Sequence models are designed to work with positions and context.

The design question is: **how should information at one position influence another?**

RNNs: Adding Memory

Recurrent Neural Networks (RNNs) process sequences one step at a time, passing a “memory” forward to the next step: a running state that summarizes everything read so far.

The fading arrows illustrate a possible difficulty with long-range information, not a fixed five-word memory limit. Each recurrent step depends on the previous state.

The Bottleneck Problem

A recurrent network carries a fixed-size state from one position to the next.

- That state must retain useful details from earlier positions.
- Long-range relationships can be difficult to learn and preserve.
- The dependence on the previous state limits parallel processing across positions.

  Attention offers a more direct route between positions. It does not need to pass every detail through a single running state.

“Attention Is All You Need” (2017)

In 2017, Google researchers published arguably the most important AI paper of the decade. The solution to RNNs forgetting and being slow? **Don’t process words one at a time at all.** They introduced the **Transformer**: built entirely on a mechanism called **attention**, with no recurrence. Every major AI model today: GPT, Claude, Gemini, LLaMA: is a transformer.

Attention Changes the Question

Attention combines information from the token positions available to the current position.

- A recurrent network passes a state forward through the sequence.
- Attention can connect directly to earlier positions within the context window.
- In a GPT-style model, a **causal mask** blocks future tokens.

  Training can process many positions in parallel while respecting that mask. Generating new text still proceeds one token at a time.

Attention: Which Tokens Can It Use?

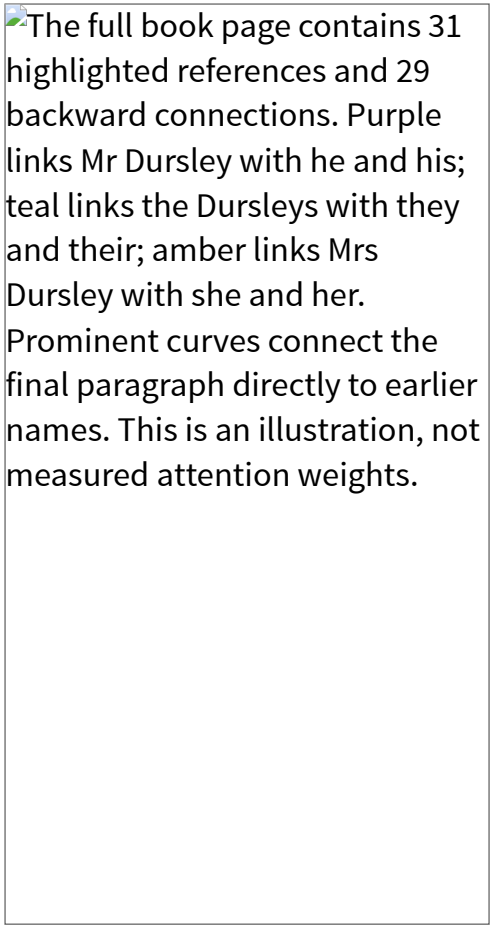
Select a position. Which tokens is it allowed to use to predict the next token?

The	cat	sat	on	the	mat
1	2	3	4	5	6

Position 3 can use: The cat sat → predict on

3 allowed positions; 3 future positions blocked. The mask prevents the model from looking at the answer during training. Each displayed word is one token in this teaching example. Click any position, including a faded one, to inspect its mask. In a GPT-style model, a position can attend to **itself and earlier positions**. Future positions are masked out. Allowed does not mean equally useful.

Attention Can Connect Distant Words



A word can draw on useful context **many sentences earlier**.

Mr Dursley · he · his
The Dursleys · they · their
Mrs Dursley · she · her
The last **“he”** can still connect to the first **“Mr Dursley”**.
The long curves show **direct connections across paragraphs**.
Attention can reach back within the **context window**.

Arrows point to earlier context. Illustrative connections, not measured attention weights. Real models use tokens.

Why This Matters: Same Word, Different Meaning

“Mole” can refer to an **animal**, a **chemistry unit**, or a **skin mark**.

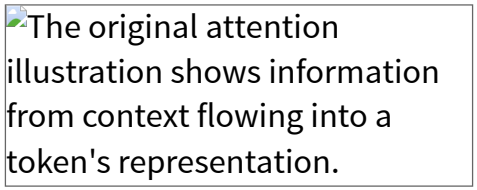
The token’s starting embedding is shared. Its representation changes as the network combines it with the available context.

Attention Updates Meaning

After attention, the embedding for “mole” has been **pulled toward the right meaning** by its context:

“American shrew” pulls “mole” toward the animal direction. “Carbon dioxide” pulls it toward the chemistry direction. Same starting point, different ending: that’s what attention does.

How Attention Combines Context



1. **Find useful context:** compare the current token with the tokens it can use.
2. **Set the mix:** give more relevant tokens a larger share.
3. **Update meaning:** combine their information with the current representation.

Image: **3Blue1Brown**

Attention Cost Matters

The price of “every word looks at every other word”: the work grows with the **square** of the length.

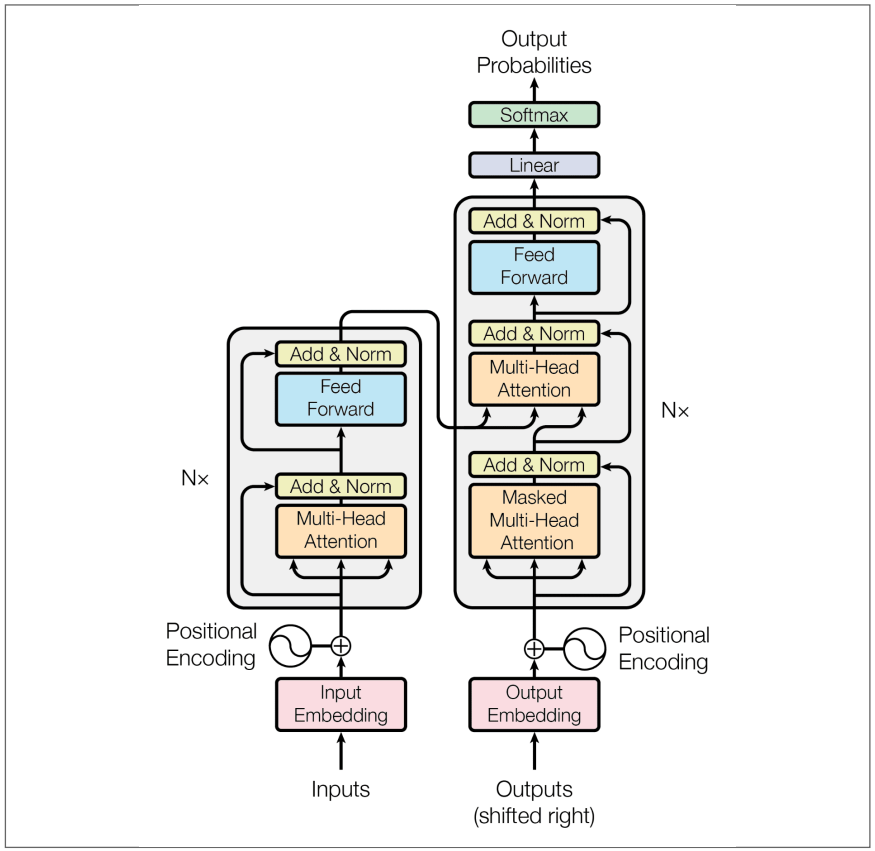
- 1,000 tokens → 1M comparisons. 100,000 tokens → **10 billion**. Doubling your document quadruples the compute
- This is *the* engineering and economic constraint behind **context windows**: the caps on how much text a model can consider at once
- It’s why long-context models cost more per request, why providers charge per token, and why “just paste the whole database into the prompt” is not an architecture

> **Note**
>
> When we discuss context windows, token economics, and retrieval in Class 5, this quadratic is the physics underneath every pricing table you’ll see.

Image: **3Blue1Brown**

Transformers

The Transformer Block



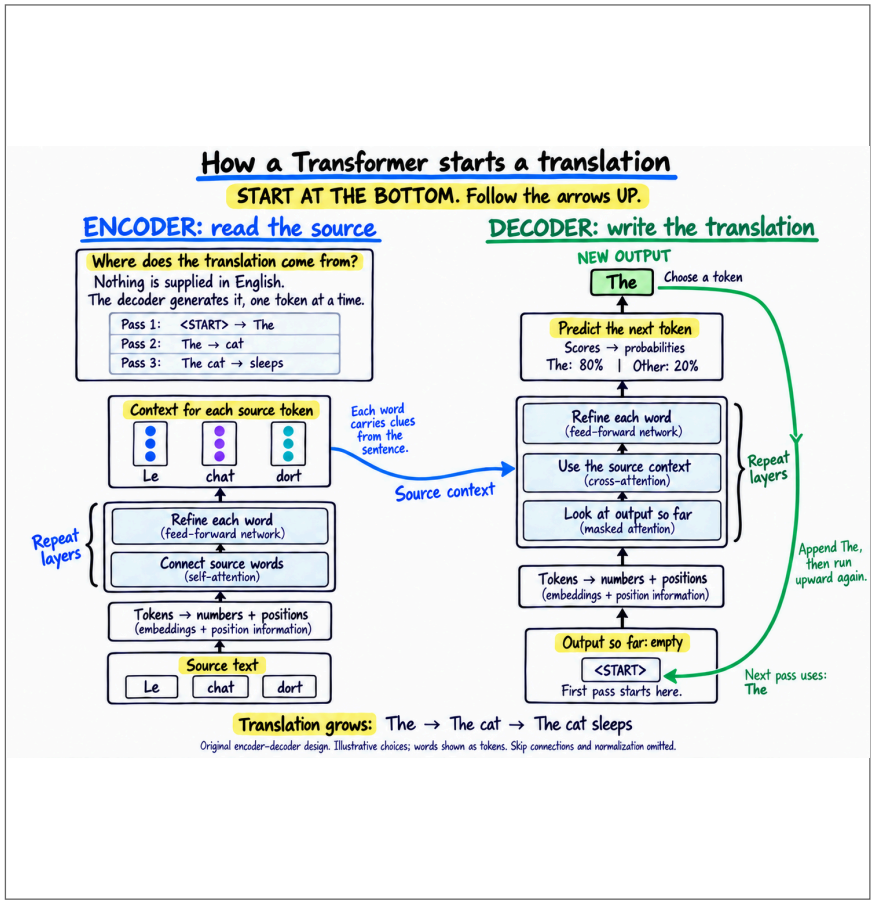
Vaswani et al., **Attention Is All You Need** (2017), Figure 1.

Read the architecture from the bottom up.

> **embeddings + positions**
> ↓
> **attention**
> ↓
> **feed-forward network**
> ↓
> **repeat**

Attention moves information between tokens.
The **feed-forward network** refines each token after attention gathers its context.
N× means the model stacks this block across many layers.

The Same Architecture, Simplified



Start at the bottom and move up.

Read the source.

The encoder turns each source token into a context-aware representation.

Start the output.

The decoder begins with a special start token. No English translation is supplied.

Use both clues.

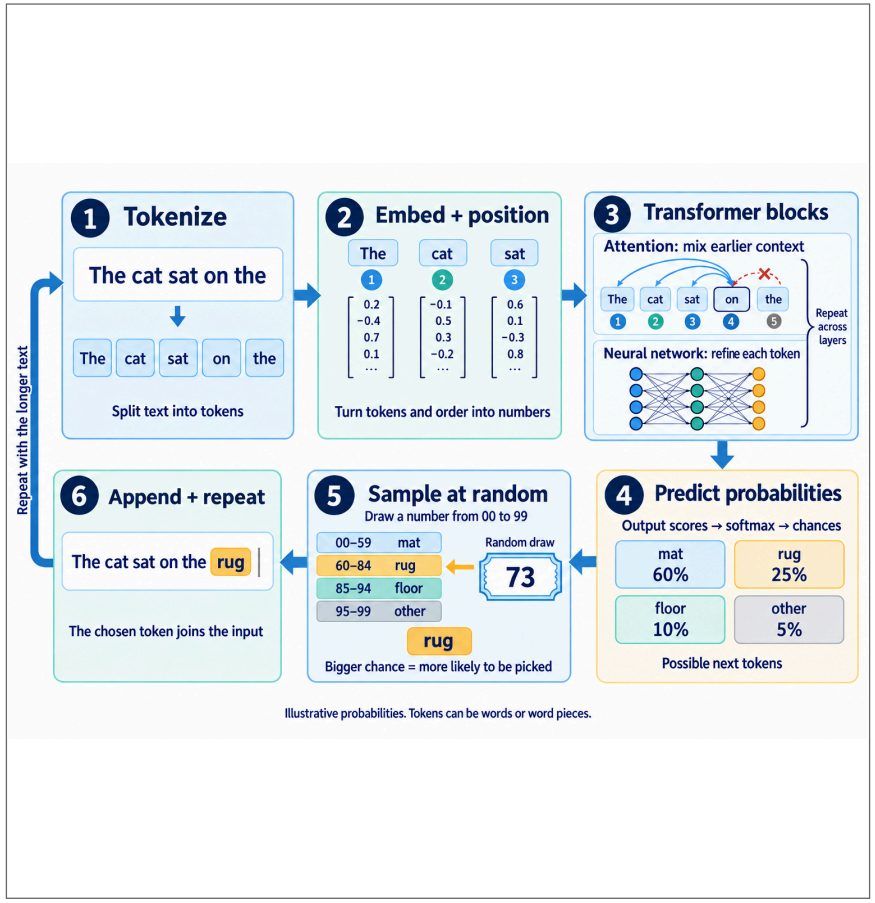
The decoder combines the output so far with the encoder's source context.

Choose and repeat.

The model predicts one token, appends it, and runs the decoder again.

> The original Transformer uses an **encoder and decoder**. GPT-style models use only the decoder side.

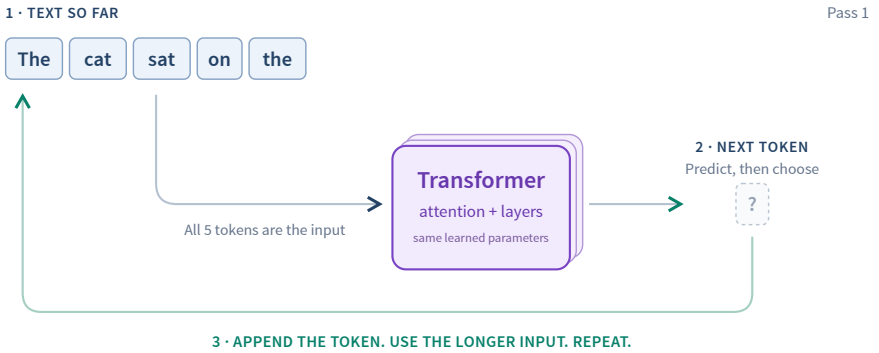
How a Transformer Processes Text



When sampling, **more likely does not mean guaranteed**.

Generating Text, One Token at a Time

Each new token becomes part of the next input.



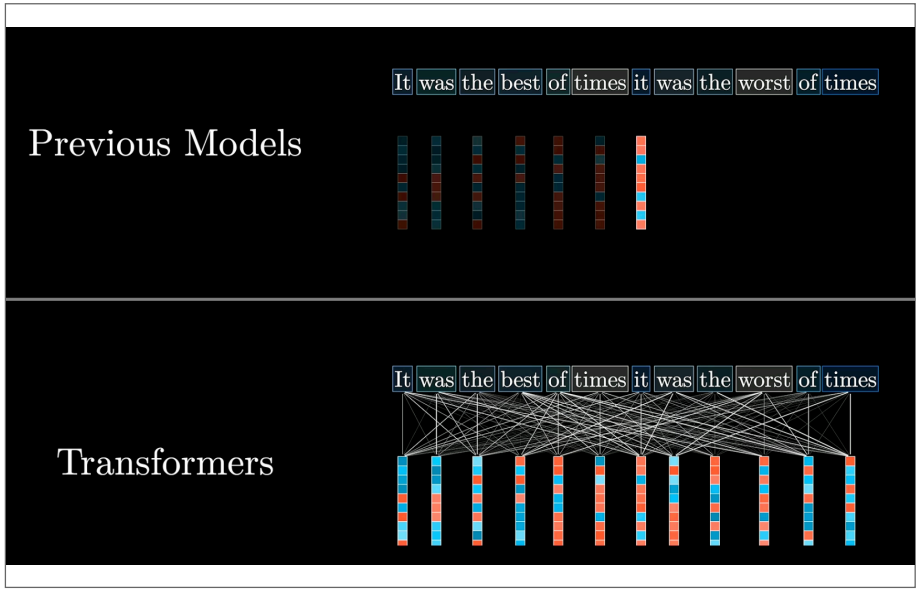
Start with the prompt. The model will add one token at a time.

Play Step Restart

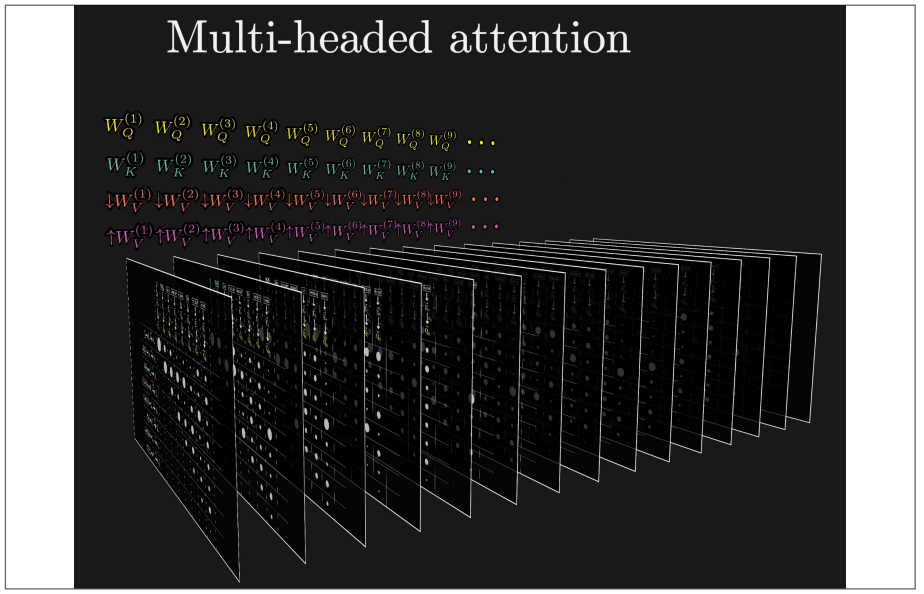
Illustrative sequence. Tokens are shown as words and punctuation; real tokens can also be word pieces.

Multi-Head Attention: Several Questions at Once

Multiple heads compute different mixtures in parallel, each with its own learned projections.



Attention connects information across tokens.

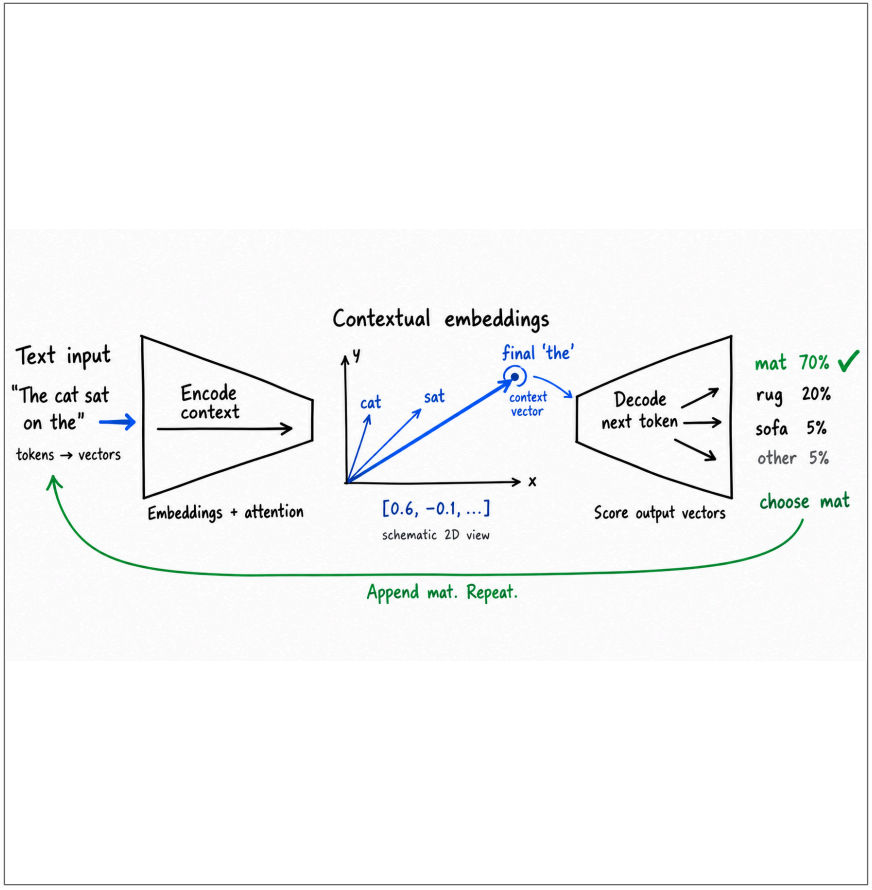


Each head learns its own set of matrices.

- Heads can capture different relationships, sometimes including syntax or reference.
- We choose the number of heads; training learns their parameters.
- Heads are not assigned neat human jobs, and attention weights alone do not fully explain a prediction.

Encode, Embed, Decode

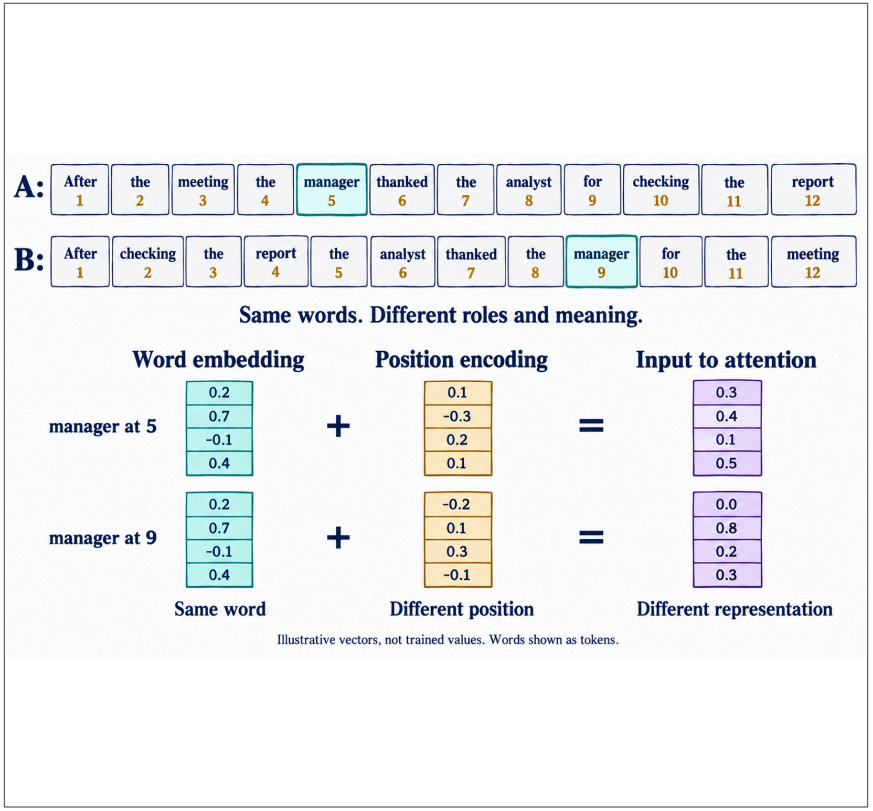
The decoder builds a **context vector**, then scores the possible **next tokens**.



Decoder layers build the **context vector**; the output layer scores the next tokens. The funnels show information flow, not literal layer sizes.

Positional Information Preserves Order

The same word needs a signal for **where it appears**, as well as **what it means**.



- RNNs tracked order one step at a time. Attention needs **position information** supplied explicitly.

- Shown here: add a **position vector** to each embedding before attention mixes the context.

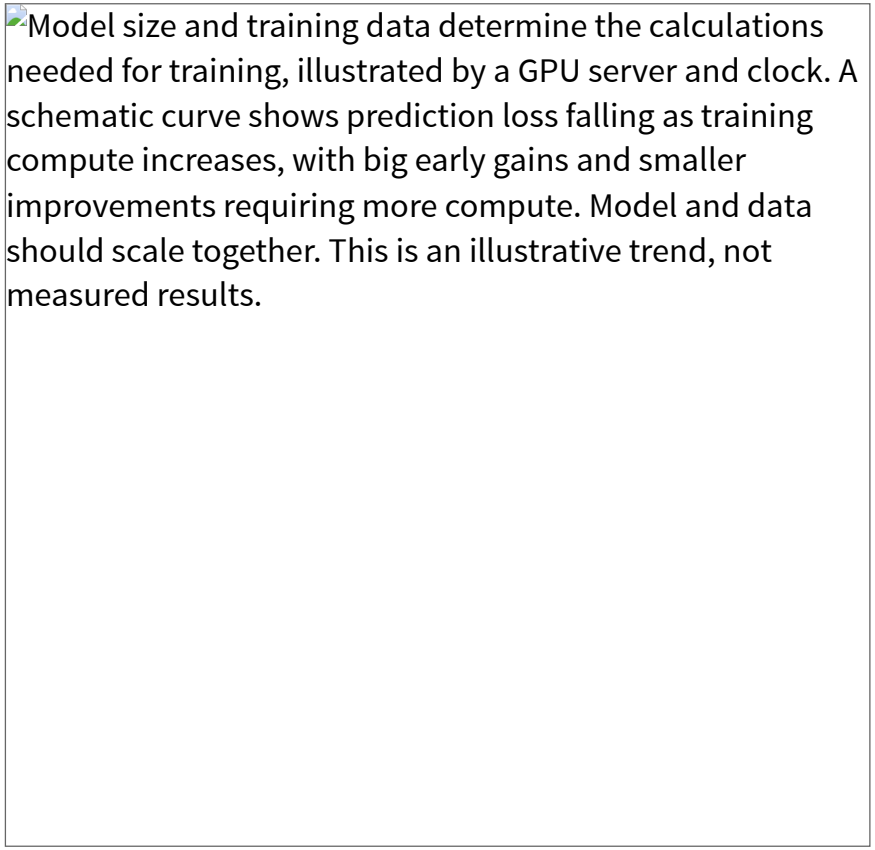
Residual Connections Keep Learning Stable

A detail that made 96-layer networks possible: each block **adds its output to its input**, rather than replacing it.

- Layers **refine an existing representation** instead of repeatedly rebuilding it from scratch: edit the memo, don't rewrite it
- It also keeps backpropagation healthy: correction signals flow backward through the "shortcut" without fading across dozens of layers (the *vanishing gradient* problem that plagued early deep networks)
- You've already seen this: the mole diagram: context is **added** to the original embedding. That plus sign is a residual connection

Scaling Laws: How Compute Improves Predictions

When model size and training data grow together, more **training compute** tends to reduce **prediction loss**.



Pretraining Learns Broad Statistical Structure

The recipe behind every foundation model:

- Take **massive text corpora**: a filtered slice of the internet, books, code
- One task, repeated trillions of times: **predict the missing or next token**
- No labels, no annotators: the text itself is the supervision. Every sentence is a tiny exam with the answer key attached
- Result: surprisingly **general** capabilities: grammar, facts, style, reasoning patterns: from a task nobody would call intelligent

> **Note**
>
> Remember the labeling bottleneck from Class 3: “the biggest cost in ML is labeled data”? Pretraining is the industry-scale workaround: text supplies its own prediction targets if your task is “guess the next token.” Data still requires collection, filtering, and permission to use. That economic unlock, as much as any architecture, is why language became AI’s center of gravity.

Transfer Learning: Don’t Start From Scratch

Reuse a pretrained network as a starting point, then adapt it to another task.

- A pretrained image network can be adapted to a smaller labeled image dataset.
- A pretrained language model can be adapted using examples of a specialized task.
- Fine-tuning changes parameters; providing context in a prompt does not.

  Pretraining can be expensive at large scale. Our tiny classroom model is small enough to train from scratch.

Fine-Tuning and Alignment Shape Behavior

A pretrained model is a text predictor, not an assistant. Two further stages shape behavior:

- **Fine-tuning**: continue training on curated examples: your domain's documents, or demonstrations of helpful question-answering
- **Alignment** (RLHF: reinforcement learning from human feedback): humans rank the model's answers; RL: the same reward-driven learning from Class 3's Grid World and Pac-Man: nudges the model toward answers people prefer
- These make a base model more **helpful, safe, or specialized**: but they do not make it omniscient. Tuning shapes *how* it answers, not *what is true*

> **Note**
>
> Class 5 opens with exactly this journey: base model → assistant: including what alignment costs, what it fixes, and the strange failure modes it leaves behind.

GPT: Predict the Next Word, at Scale

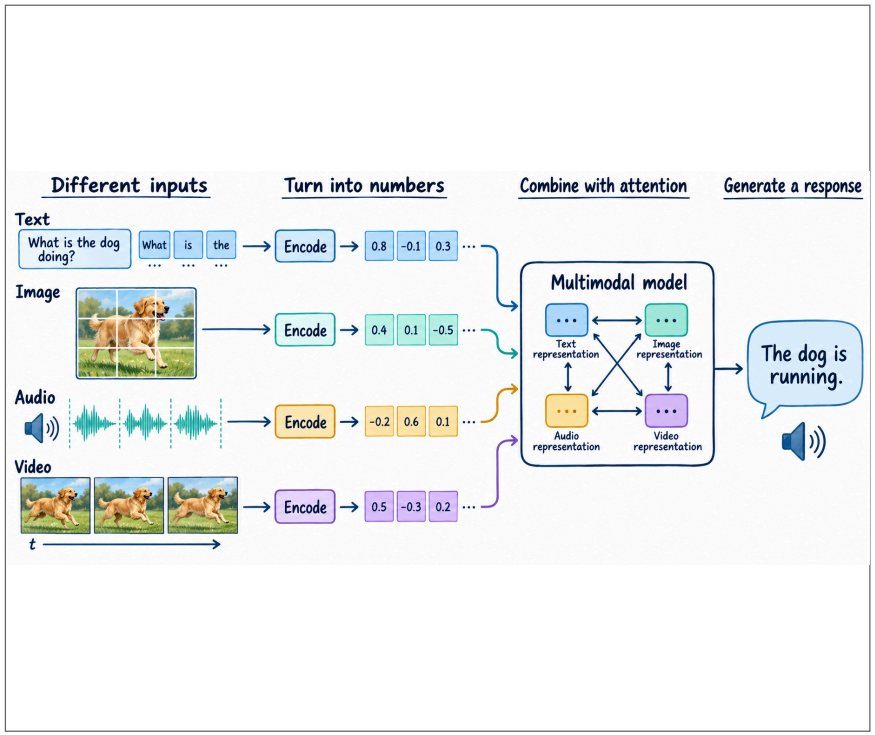
GPT (Generative Pre-trained Transformer) has one training objective: **predict the next token**. Given "The cat sat on the," predict "mat." That's it: billions of times, on billions of documents.

From this simple objective, the model learns grammar, facts, reasoning, and coding: all as a side effect of getting better at prediction.

> **Note**
>
> GPT-1 (2018) had 117 million parameters. GPT-5 (2025) has an estimated 5 **trillion**. Same idea, 40,000x bigger: and the results went from "interesting toy" to "writes better than most humans."

Beyond Text: The Same Machine, More Senses

Multimodal means working with more than one kind of data. Different inputs become **vectors**: lists of numbers that attention can combine.



Same attention idea, different input encoders. Supported inputs and outputs depend on the model; numbers are illustrative.

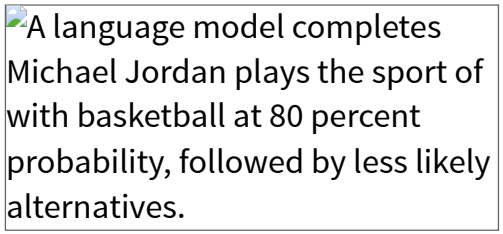
> **Why it matters:** Ask a question about a screenshot, discuss a recording, or describe an image to generate. **Context limits, hallucinations, and processing costs still matter.**

AI Progress Since 2022

2022	2023	2024	2025	2026
Converse	**See & code**	**Reason**	**Do the work**	**Discover**
ChatGPT brings natural conversation, writing, and explanations to everyone.	GPT-4 interprets images and handles more complex coding tasks.	Models spend more time working through difficult math and science problems.	Coding agents edit files, run tests, and complete delegated tasks.	AI contributes new mathematics, including a disproof of an Erdős conjecture.
ChatGPT ↗	**GPT-4 ↗**	**OpenAI o1 ↗**	**Codex ↗**	**New mathematics ↗**

A Fluent Answer Is Still a Prediction

Language-model training rewards predicting text. A confident prediction can be correct, but confidence does not verify the answer.



- Prediction can recover useful facts and patterns from training data.
- It can also produce plausible names, dates, or citations that are wrong.
- Retrieval and tools can reduce errors. **Evidence verifies important claims.**

Next class, we connect models to retrieved information and evaluate whether it improves their answers.

The 2026 Model Landscape

Providers now offer a **portfolio of models** for different levels of quality, speed, cost, and control.

Part of the market	Current examples	When it matters
Frontier generalists	GPT-6 Astra, Claude 5, Gemini 3, Grok 4.6	Complex analysis, coding, and agent workflows
Efficient hosted models	GPT-5.6 Luna, Gemini Flash / Flash-Lite, DeepSeek Flash	High-volume work where latency and cost matter
Open-weight models	gpt-oss, Llama 4, Qwen3, Mistral Large 3	Private deployment, customization, and infrastructure control
Specialist models	Voice, image, video, coding, and research models	A specific modality or workflow matters more than broad capability

> **Tip**

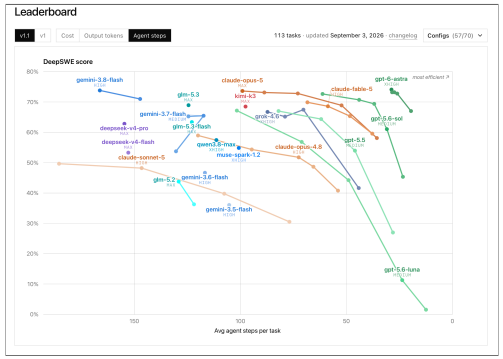
> **Evaluate models on your own work.** Public leaderboards cannot measure your data policy, integration effort, latency, or total cost.

Snapshot: September 2026. Model names and availability change quickly.

Reading Model Leaderboards

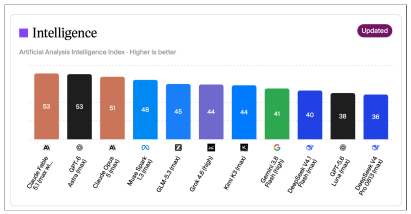
Each leaderboard tests a different slice of model performance.

DeepSWE



Coding agents: 113 original, long-horizon engineering tasks. Compare task score with cost, output tokens, or agent steps.

Artificial Analysis

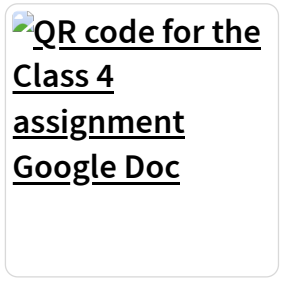


Broader model comparison: an Intelligence Index built from ten evaluations, with separate views of speed and cost.

> **Tip**
>
> **A leaderboard measures its test.** The final comparison should use your own tasks and constraints.

Assignment 3: Train Your Own Tiny LLM

1. Assignment Doc ↗
Read the requirements.



Scan for the assignment

2. Code Repository ↗
Get the starter and examples.

3. Open in Colab ↗
Save a copy and run it.

Your task: choose text, train the supplied tiny language model, and explain what changed using your own outputs.

- **Start with:** the classroom corpus, **3,000 steps**, learning rate **0.001**.

- **Hand in:** one public GitHub repository with your executed notebook, results, and README.

  The notebook supplies the model. A CPU is enough; no model API is needed.

Run It, Then Inspect What Changed

1. **Choose and predict.** Use the classroom text or add your own corpus. Set the steps and learning rate; write what you expect to happen.
2. **Train.** Try **10 steps** to check setup, then **3,000** as a starting budget. Select **Run All** and let the notebook finish.
3. **Compare.** Look at the untrained, halfway, and final samples, both loss curves, and the temperature comparison.

> **Tip**
>
> **Follow one real example:** word → token ID → embedding → next-token prediction. Show one gradient and weight update, then compare the vector and prediction after training.

Keep the data split and baseline generation settings fixed so the comparison is fair.

Submit One Repository with Visible Evidence

- **Executed notebook:** save after the final run, with all outputs visible.
- **Results:** before / halfway / final samples, training + validation loss plot and table, and the temperature comparison.
- **README:** record your corpus and settings; use actual values to explain tokens, embeddings, attention, predictions, and a weight update. Finish with **one limitation and one next experiment**.

  In Colab: download the results ZIP **and** the executed notebook separately. Add the notebook, README, and selected evidence to your public repository.

  Submit your GitHub repository URL through the Class 4 assignment on bCourses.

Thank You & What’s Next

Thank you for being here: giving up your personal time to learn this. It matters, and it will pay off.
You have seen the pieces of a language model: neurons, learning, embeddings, attention, and transformers. The assignment connects those pieces in a tiny model you can train and inspect. Next class we use larger models: why prompts work, why they fail, how context windows behave, and how retrieval supplies information a model can cite. Your notebook’s temperature comparison is where we pick up.

Made with Magic by 3Blue1Brown

Speaker notes